# Projekt zaliczeniowy

Omówienie SSL/TLS pod katem bezpieczeństwa. Działanie i sposoby podsłuchiwania oraz bezpieczna wymiana informacji.

**Autorzy**
Krystian Maj (71482)
Michał Michalak (71307)
Nikodem Faber (71070)

**1. Czym jest kryptografia.....2**
**2. Czym jest szyfrowanie.....6**
**3. Jakie są dostępne metody szyfrowania.....8**
**4. Zasady działania szyfrowania.....14**
**5. Czym jest i jak działa SSL.....17**
**6. Czym jest i jak działa TLS.....22**
**7. Różnice pomiędzy SSL a TLS.....32**
**8. Metody nasłuchiwania, programy.....34**
**9. Opis stworzonej infrastruktury.....46**
9.1. Docker.....46
9.2. Nginx.....47
9.3. Protokoły HTTP.....48
9.4. Protokół POST, zasady jego działania.....49
**10. Opis doświadczenia.....49**
10.1. Pozyskiwanie certyfikatu.....50
10.2. Implementacja certyfikatu.....51
10.3. Przechwytywanie danych.....52
10.3.1. Bez zabezpieczenia.....52
10.3.2. Z zabezpieczeniem.....54
10.3.3. Z zabezpieczeniem, ale posiadając klucze certyfikatu.....55
10.4. Podsumowanie.....56
**11. Instrukcja jak odtworzyć doświadczenie.....56**

# 1. Czym jest kryptografia

Kryptografia to gałąź nauki, która zajmuje się zabezpieczaniem informacji poprzez ich szyfrowanie, czyli przekształcanie w sposób nieczytelny dla osób nieupoważnionych, oraz odszyfrowywanie, czyli przywracanie do postaci oryginalnej. Jej istotnym celem jest zapewnienie poufności danych, czyli zachowanie ich tajności przed nieuprawnionym dostępem. Jednakże, kryptografia nie ogranicza się jedynie do poufności, ale również zapewnia integralność danych (czyli zapobieganie ich modyfikacji lub uszkodzeniu) oraz autentyczność (czyli potwierdzanie tożsamości nadawcy i odbiorcy danych).

W praktyce, proces szyfrowania polega na wykorzystaniu algorytmu kryptograficznego, który jest matematyczną procedurą określającą sposób przekształcenia danych, oraz klucza, który jest parametrem tego algorytmu. Klucz determinuje sposób szyfrowania i deszyfrowania danych, a jego rodzaj zależy od wybranej metody kryptograficznej. Istnieją dwie główne kategorie kluczy kryptograficznych: klucze symetryczne, gdzie ten sam klucz jest używany do szyfrowania i deszyfrowania danych oraz klucze asymetryczne (znane również jako klucze publiczne i prywatne), gdzie para różnych kluczy jest używana do szyfrowania i deszyfrowania danych.

Oprócz samego szyfrowania, kryptografia obejmuje również szeroki zakres innych zagadnień, takich jak zarządzanie kluczami (czyli generowanie, dystrybucja, przechowywanie i wykorzystanie kluczy kryptograficznych), protokoły kryptograficzne (które definiują zbiory reguł i procedur do bezpiecznej komunikacji) oraz algorytmy kryptograficzne (które są matematycznymi technikami wykorzystywanymi do szyfrowania i deszyfrowania danych).

Współczesna kryptografia znajduje zastosowanie w różnych dziedzinach, takich jak bezpieczeństwo sieci komputerowych, komunikacja telekomunikacyjna, bankowość elektroniczna, e-commerce, ochrona danych osobowych, bezpieczeństwo transakcji finansowych, czy nawet ochrona danych medycznych. Jej rozwój i zastosowanie jest kluczowe w obliczu rosnących zagrożeń cybernetycznych oraz coraz większej ilości danych przesyłanych i przechowywanych w środowisku cyfrowym.

Oprócz głównych zastosowań, kryptografia ma także istotne znaczenie w dziedzinie ochrony danych osobowych. Wraz z wprowadzeniem rozporządzenia ogólnego o ochronie danych osobowych (RODO) w Unii Europejskiej oraz podobnych regulacji na całym świecie, kwestie związane z bezpieczeństwem danych stały się kluczowym elementem dla organizacji i przedsiębiorstw. Kryptografia odgrywa zatem ważną rolę w zapewnieniu zgodności z przepisami dotyczącymi ochrony prywatności poprzez zabezpieczenie danych osobowych przed nieuprawnionym dostępem.

Ponadto, kryptografia jest niezbędna w kontekście bezpieczeństwa sieci komputerowych oraz komunikacji internetowej. Wraz z rozwojem technologii, coraz więcej danych jest przesyłanych przez sieci, co niesie za sobą ryzyko przechwycenia i wykorzystania przez osoby nieuprawnione. Szyfrowanie danych jest zatem nieodzownym elementem w zapewnieniu poufności i bezpieczeństwa komunikacji online oraz w transmisji danych w sieciach komputerowych.

Ponadto, rosnące znaczenie kryptografii jest związane również z pojawieniem się nowych technologii i trendów, takich jak Internet rzeczy (IoT), sztuczna inteligencja (AI) czy analiza big data. Wraz z coraz większą ilością danych generowanych i przetwarzanych przez te technologie, konieczne staje się stosowanie zaawansowanych metod szyfrowania, aby chronić prywatność i bezpieczeństwo użytkowników oraz danych.

W kontekście globalnej gospodarki cyfrowej oraz coraz większej liczby interakcji online, kryptografia będzie odgrywać coraz większą rolę w zapewnieniu bezpieczeństwa, prywatności i integralności danych. Jej znaczenie będzie się tylko zwiększać wraz z rozwojem technologicznym i rosnącym zapotrzebowaniem na bezpieczne sposoby przechowywania i przesyłania informacji w cyfrowym świecie.

Oprócz zastosowań w dziedzinie ochrony danych osobowych i bezpieczeństwa sieci komputerowych, kryptografia ma również liczne inne zastosowania w codziennym życiu oraz w różnych branżach.

1. **Bezpieczne transakcje finansowe**: W sektorze bankowości i finansów, kryptografia jest niezbędna do zapewnienia bezpieczeństwa transakcji online, dostępu do kont bankowych oraz przechowywania danych finansowych klientów. Protokoły kryptograficzne, takie jak SSL/TLS, są wykorzystywane w serwisach bankowości internetowej oraz platformach płatności online, aby zapewnić poufność i integralność transakcji.

2. **Bezpieczna komunikacja korporacyjna**: W biznesie, kryptografia jest wykorzystywana do zapewnienia bezpieczeństwa komunikacji korporacyjnej, takiej jak e-maile biznesowe, wideokonferencje oraz komunikatory internetowe. Firmy często korzystają z szyfrowania end-to-end, aby zabezpieczyć poufność swoich informacji przed atakami cybernetycznymi oraz nieuprawnionym dostępem.

3. **Ochrona danych medycznych**: W sektorze opieki zdrowotnej, kryptografia odgrywa istotną rolę w ochronie danych medycznych pacjentów. Szyfrowanie danych medycznych jest niezbędne do zapewnienia zgodności z przepisami dotyczącymi prywatności, takimi jak HIPAA w Stanach Zjednoczonych. Przykładowo, dane pacjentów przechowywane w systemach informatycznych szpitali i klinik muszą być szyfrowane, aby zapobiec nieautoryzowanemu dostępowi i utracie poufności informacji medycznych.

4. **Bezpieczne połączenia internetowe w sektorze publicznym**: W administracji publicznej, kryptografia jest wykorzystywana do zapewnienia bezpiecznych połączeń internetowych oraz ochrony danych obywateli. Przykładowo, strony internetowe rządowe, portale administracyjne oraz platformy e-usług często korzystają z protokołów kryptograficznych, takich jak HTTPS, aby zapewnić bezpieczne przesyłanie danych osobowych obywateli, takich jak informacje podatkowe czy dokumenty tożsamości.

5. **Bezpieczeństwo danych w chmurze**: Wraz z rosnącym trendem korzystania z usług chmurowych, kryptografia odgrywa kluczową rolę w zapewnieniu bezpieczeństwa danych przechowywanych w chmurze. Firmy chmurowe stosują zaawansowane metody szyfrowania, takie jak AES, do zabezpieczenia danych klientów przechowywanych w chmurze obliczeniowej, co zapewnia ochronę przed nieautoryzowanym dostępem i utratą poufności informacji.

Oprócz wymienionych zastosowań, kryptografia ma także znaczenie w dziedzinie bezpieczeństwa narodowego i obronności. Państwa oraz organizacje odpowiedzialne za bezpieczeństwo narodowe wykorzystują zaawansowane metody szyfrowania do ochrony poufnych informacji, komunikacji wojskowej, oraz przesyłania danych wywiadowczych. Przykładowo, szyfrowane komunikatory są wykorzystywane przez siły zbrojne do bezpiecznej wymiany informacji w terenie oraz do zapewnienia poufności operacji wojskowych.

Kryptografia odgrywa także istotną rolę w ochronie własności intelektualnej i tajemnic handlowych w środowisku biznesowym. Firmy korzystają z szyfrowania, aby zabezpieczyć swoje tajemnice handlowe, plany rozwoju produktów oraz inne wrażliwe informacje przed konkurencją oraz przedsiębiorstwami szpiegującymi. Bezpieczne przechowywanie i przesyłanie danych firmowych za pomocą kryptografii jest niezbędne w dzisiejszym środowisku biznesowym, gdzie konkurencja jest zaciekła, a cyberzagrożenia stale rosną.

W kontekście przyszłościowej kryptografii, istnieje również coraz większe zainteresowanie rozwojem metod kryptografii kwantowej. Kryptografia kwantowa wykorzystuje prawa mechaniki kwantowej do zapewnienia absolutnie bezpiecznej komunikacji, niezależnej od obecnego stanu technologii obliczeniowych. Chociaż kryptografia kwantowa jest obecnie w fazie badań naukowych i eksperymentów laboratoryjnych, jej potencjalne zastosowania w przyszłości mogą całkowicie zmienić oblicze bezpieczeństwa informatycznego.

Podsumowując, kryptografia jest niezwykle wszechstronną dziedziną, której znaczenie stale rośnie wraz z rozwojem technologii cyfrowych oraz zwiększającym się zapotrzebowaniem na bezpieczne sposoby przechowywania i przesyłania informacji w świecie zdominowanym przez internet. Jej zastosowania obejmują obszary od ochrony prywatności osobistej po bezpieczeństwo narodowe, a jej rola w zapewnianiu bezpieczeństwa danych jest kluczowa dla funkcjonowania współczesnych społeczeństw i gospodarek.

# 2. Czym jest szyfrowanie

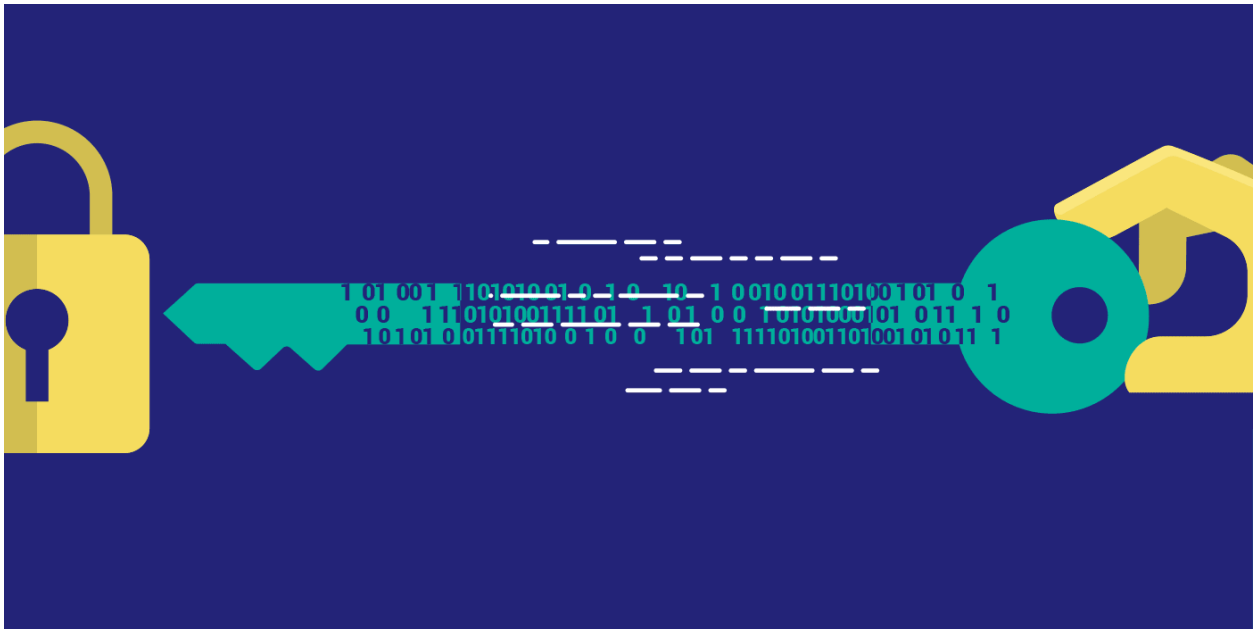
Szyfrowanie to proces przekształcania czytelnych danych, nazywanych tekstem jawnym, w sposób niezrozumiały dla osób nieupoważnionych, co tworzy tzw. tekst zaszyfrowany. Jest to kluczowy mechanizm stosowany w dziedzinie kryptografii do zachowania poufności, integralności i autentyczności przesyłanych informacji.

Istnieją różne metody szyfrowania, ale zasada działania większości z nich opiera się na użyciu algorytmu oraz klucza. Algorytm to zestaw instrukcji i procedur matematycznych, które określają sposób przekształcenia danych z postaci czytelnej na nieczytelną. Klucz natomiast jest parametrem tego algorytmu i determinuje sposób szyfrowania i deszyfrowania danych. W przypadku szyfrowania symetrycznego, ten sam klucz jest używany do zarówno szyfrowania, jak i deszyfrowania danych, podczas gdy w

szyfrowaniu asymetrycznym używane są pary kluczy: publiczny do szyfrowania i prywatny do deszyfrowania.

Podstawowe zasady działania szyfrowania obejmują:

1. **Poufność:** Szyfrowanie zapewnia poufność danych poprzez utrudnienie dostępu do treści przez osoby nieuprawnione. Tekst zaszyfrowany jest niemożliwy do zrozumienia bez użycia odpowiedniego klucza deszyfrującego.
2. **Integralność:** Szyfrowanie chroni integralność danych, zapobiegając ich modyfikacji lub manipulacji przez osoby trzecie podczas transmisji lub przechowywania.
3. **Autentyczność:** Szyfrowanie może również zapewnić autentyczność danych, umożliwiając odbiorcy potwierdzenie, że dane pochodzą od rzeczywistego nadawcy i nie zostały zmienione w trakcie transmisji.



Szyfrowanie jest niezwykle istotnym mechanizmem używanym w dziedzinie kryptografii do ochrony danych przed nieautoryzowanym dostępem oraz zapewnienia bezpieczeństwa komunikacji. Jego istota polega na przekształceniu danych w sposób, który uniemożliwia ich zrozumienie lub odczytanie przez osoby nieuprawnione, tworząc tzw. tekst zaszyfrowany. Proces szyfrowania polega na zastosowaniu algorytmu kryptograficznego, który operuje na danych wejściowych (tekście jawnym) przy użyciu określonego klucza, co prowadzi do wygenerowania zaszyfrowanych danych (tekstu zaszyfrowanego).

Współczesne metody szyfrowania mogą być podzielone na dwie główne kategorie: szyfrowanie symetryczne i asymetryczne. W szyfrowaniu symetrycznym ten sam klucz jest używany zarówno do szyfrowania, jak i deszyfrowania danych, co oznacza, że nadawca i odbiorca muszą mieć dostęp do tego samego klucza. Natomiast w szyfrowaniu asymetrycznym używane są pary kluczy: publiczny do szyfrowania i prywatny do deszyfrowania, co umożliwia bezpieczną komunikację bez konieczności współdzielenia klucza.

Szyfrowanie odgrywa kluczową rolę w zachowaniu poufności danych, szczególnie w przypadku przesyłania informacji przez sieci komputerowe, gdzie istnieje ryzyko przechwycenia danych przez osoby trzecie. Jest również wykorzystywane w różnych obszarach, takich jak bankowość elektroniczna, ochrona danych osobowych, bezpieczeństwo sieci komputerowych oraz komunikacja telekomunikacyjna.

Współczesne metody szyfrowania są również stale rozwijane wraz z postępem technologicznym i ewolucją zagrożeń cybernetycznych. Nowe algorytmy kryptograficzne są opracowywane w celu zwiększenia bezpieczeństwa danych oraz dostosowania się do coraz bardziej zaawansowanych ataków. Wraz z rozwojem technologii kwantowych, kryptografia kwantowa staje się także obiektem intensywnych badań, dążących do stworzenia metod szyfrowania odpornych na przyszłe wyzwania związane z rozwojem technologicznym.

W podsumowaniu, szyfrowanie jest fundamentalnym narzędziem w dziedzinie kryptografii, które umożliwia bezpieczną komunikację oraz ochronę danych w świecie cyfrowym. Jego rola jest kluczowa dla zapewnienia prywatności, integralności i autentyczności danych w różnych dziedzinach życia oraz biznesu.

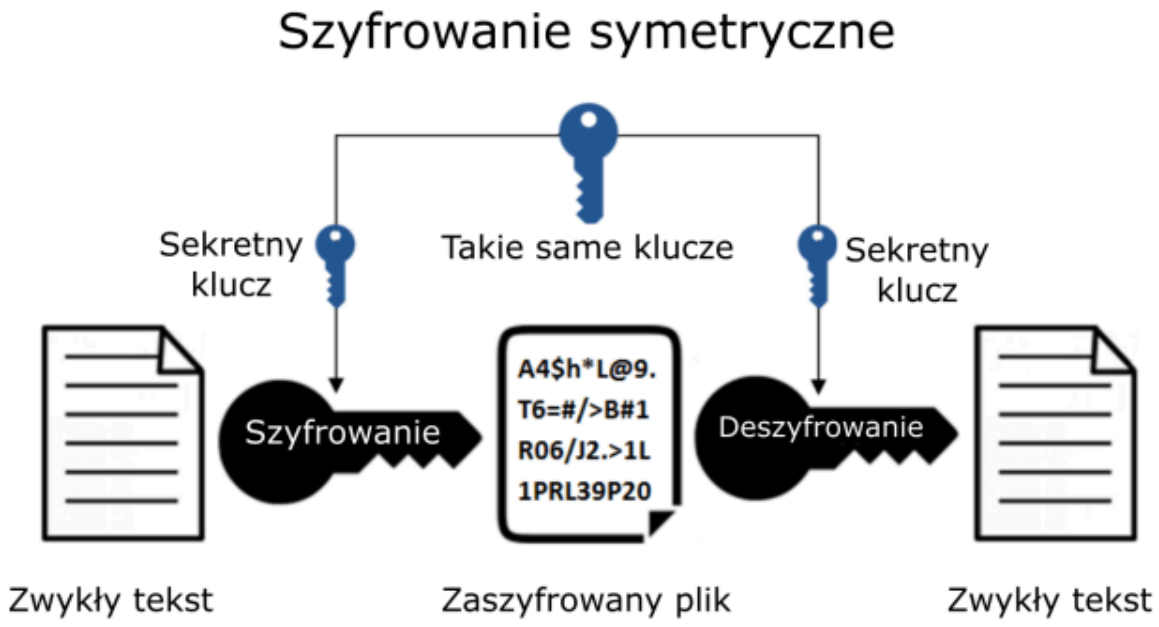
# 3. Jakie są dostępne metody szyfrowania

Istnieje wiele różnych metod szyfrowania, z których każda ma swoje unikalne cechy, zastosowania i poziom bezpieczeństwa. Poniżej przedstawiam rozbudowaną listę kilku głównych metod szyfrowania wraz z przykładami:

**Szyfrowanie symetryczne:**

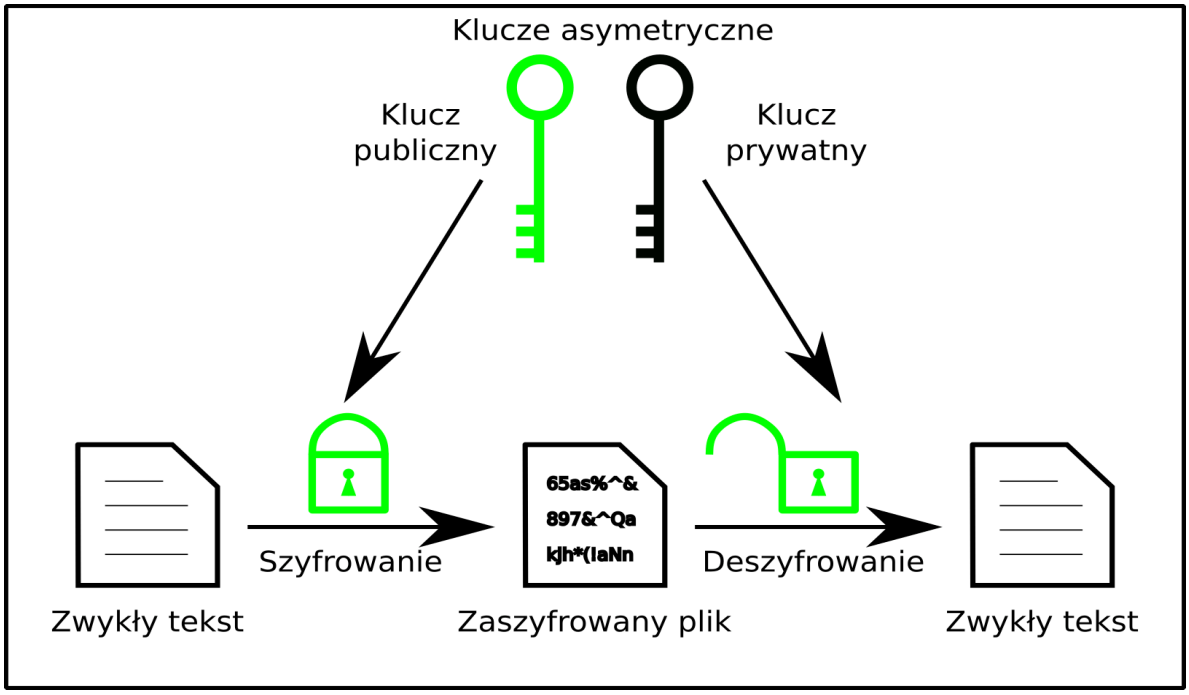
- Metoda, w której zarówno szyfrowanie, jak i deszyfrowanie danych odbywa się przy użyciu tego samego klucza.

- Przykładem algorytmu szyfrowania symetrycznego jest AES (Advanced Encryption Standard), który jest szeroko stosowany w różnych systemach informatycznych i sieciach komputerowych.



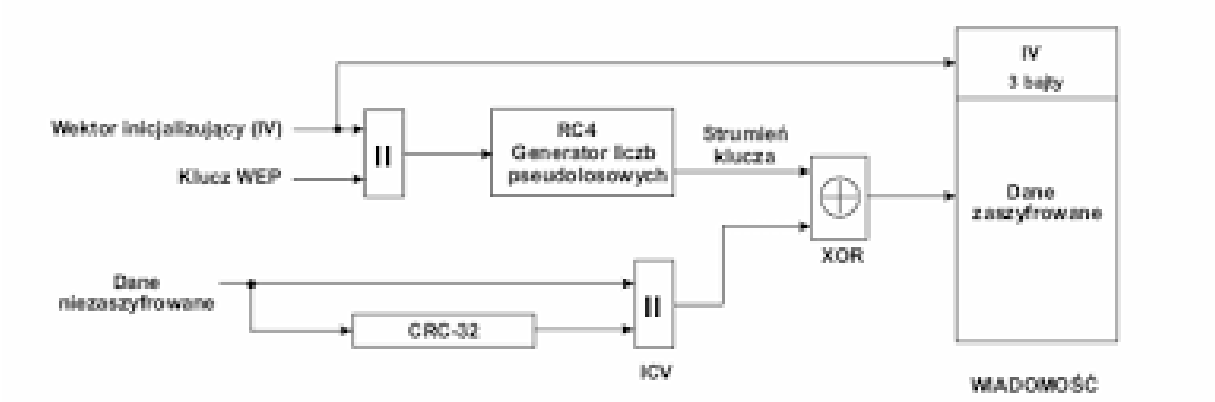

**Szyfrowanie asymetryczne:**

- Inaczej znane jako kryptografia klucza publicznego, używa dwóch różnych kluczy: publicznego do szyfrowania i prywatnego do deszyfrowania danych.
- Algorytmy takie jak RSA (Rivest-Shamir-Adleman) są przykładem szyfrowania asymetrycznego i są wykorzystywane w bezpiecznych połączeniach internetowych, takich jak HTTPS.




**Szyfrowanie strumieniowe:**

- Jest to metoda, w której dane są szyfrowane bit po bicie lub bajt po bajcie za pomocą klucza.
- Algorytmy szyfrowania strumieniowego, takie jak RC4 (Rivest Cipher 4), są stosowane w protokołach bezpiecznych komunikacji, takich jak WEP (Wired Equivalent Privacy) i WPA (Wi-Fi Protected Access).




**Szyfrowanie homomorficzne:**

- To zaawansowana technika, która umożliwia przetwarzanie danych w postaci zaszyfrowanej, pozostawiając je w zaszyfrowanej postaci i umożliwiając operacje matematyczne na zaszyfrowanych danych.
- Jest to przydatne w dziedzinach, gdzie prywatność danych jest kluczowa, takich jak ochrona danych medycznych i przetwarzanie w chmurze.

**Szyfrowanie kwantowe:**

- Opiera się na wykorzystaniu właściwości cząstek kwantowych do tworzenia kluczy kryptograficznych i przesyłania danych w sposób, który jest teoretycznie odporny na ataki kryptograficzne współczesnych komputerów.
- Jednym z przykładów jest protokół BB84, który wykorzystuje zasady mechaniki kwantowej do bezpiecznego przesyłania kluczy kryptograficznych.

**Szyfrowanie homomorficzne cząstkowe:**

- Jest to modyfikacja szyfrowania homomorficznego, która pozwala na wykonywanie operacji na częściach danych w postaci zaszyfrowanej.
- Ten rodzaj szyfrowania jest wykorzystywany w systemach przetwarzania danych w chmurze, gdzie chcemy zachować poufność danych, ale jednocześnie umożliwić przetwarzanie na zaszyfrowanych danych.

**Szyfrowanie z użyciem funkcji skrótu:**

- Polega na przekształceniu danych na podstawie funkcji skrótu, która generuje krótki, unikalny identyfikator dla danej treści.
- Jest szeroko stosowane w celu weryfikacji integralności danych, a także w bezpiecznych systemach uwierzytelniania, takich jak hashowanie haseł.

Te metody szyfrowania różnią się pod względem złożoności, wydajności, a także odporności na różne rodzaje ataków. Wybór odpowiedniej metody szyfrowania zależy od konkretnego zastosowania, wymagań bezpieczeństwa oraz dostępnych zasobów obliczeniowych.

Oprócz wymienionych metod, istnieje wiele innych technik szyfrowania oraz ich wariantów, które są stosowane w różnych kontekstach w celu zapewnienia bezpieczeństwa danych. Niektóre z tych technik to:

**Szyfrowanie jednorazowe (One-Time Pad):** Jest to metoda szyfrowania symetrycznego, w której klucz używany do szyfrowania jest tak długi jak sama wiadomość i jest wykorzystywany tylko raz. Jest to jedna z nielicznych metod, która jest teoretycznie nieodporna na ataki kryptoanalizy, jednak wymaga ona klucza o długości co najmniej takiej jak sama wiadomość, co sprawia, że jest praktycznie trudna do zastosowania w rzeczywistych scenariuszach.

**Algorytm z kluczem jednorazowym**

Algorytmem z kluczem jednorazowym nazywany jest sposób szyfrowania, gdzie do celu zaszyfrowania / odszyfrowania pojedynczego komunikatu jest stosowany za każdym razem inny klucz (o długości równej długości szyfrowanego komunikatu). Przykład:

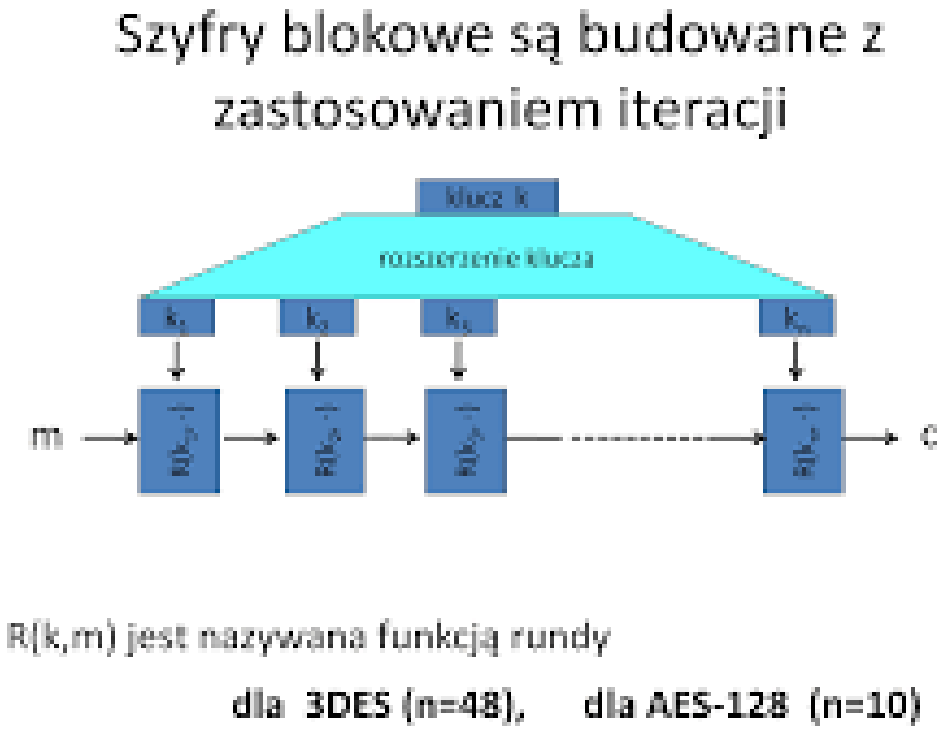
| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Komunikat: | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| Klucz: | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| Szyfrogram (suma modulo 2): | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |

Takie szyfry są stosowane do przesyłania szczególnie tajnych, dość krótkich informacji. Klucze jednorazowe po użyciu powinny być zniszczone zarówno po stronie nadawcy, jak i po stronie odbiorcy (ich wpadnięcie w ręce przeciwnika może spowodować nie tylko odczytanie zaszyfrowanych nimi komunikatów, ale również odkrycie ogólnej zasady działania generatora liczb pseudolosowych, co z kolei umożliwi odczytywanie wszystkich kolejnych komunikatów).

**Szyfrowanie z użyciem tablic (Tabular Transposition):** Metoda polegająca na przekształceniu tekstu jawnego poprzez zapisanie go w tabeli o określonej liczbie kolumn, a następnie zaszyfrowanie poprzez zmianę kolejności kolumn w tabeli według ustalonego klucza. Jest to technika szyfrowania symetrycznego, która może być stosowana do szyfrowania tekstu o stałej długości.

**Szyfrowanie dwufazowe (Two-Phase Encryption):** Metoda, która polega na zastosowaniu dwóch różnych metod szyfrowania w dwóch kolejnych fazach. Na przykład, tekst jawnego może być najpierw szyfrowany za pomocą szyfrowania symetrycznego, a następnie zaszyfrowany ponownie za pomocą szyfrowania asymetrycznego. Jest to jedna z technik stosowana w celu zwiększenia bezpieczeństwa komunikacji.

**Szyfrowanie blokowe (Block Cipher):** Metoda szyfrowania, w której dane są dzielone na bloki o stałej długości, które są następnie szyfrowane niezależnie od siebie. Szyfrowanie blokowe jest powszechnie stosowane w różnych algorytmach szyfrowania symetrycznego, takich jak DES (Data Encryption Standard) czy AES (Advanced Encryption Standard).




**Szyfrowanie oparte na permutacjach (Permutation Cipher):** Metoda szyfrowania, w której kolejność znaków w tekście jawnym jest zmieniana według określonego klucza permutacji. Jest to prosty rodzaj szyfrowania, który może być stosowany w przypadkach, gdy nie są wymagane zaawansowane mechanizmy szyfrowania.

**Szyfrowanie homomorficzne częściowe (Partially Homomorphic Encryption):** Jest to odmiana szyfrowania homomorficznego, która pozwala na wykonywanie tylko pewnych operacji matematycznych na danych zaszyfrowanych, np. dodawanie lub mnożenie, ale nie oba jednocześnie.

Te techniki są tylko kilkoma przykładami z szerokiego spektrum metod szyfrowania stosowanych w praktyce. Wybór odpowiedniej metody zależy od specyfiki aplikacji, poziomu bezpieczeństwa wymaganego przez dane oraz dostępnych zasobów obliczeniowych.

# 4. Zasady działania szyfrowania

Szyfrowanie jest niezwykle istotnym narzędziem w dzisiejszym świecie cyfrowym, gdzie transmisja i przechowywanie danych odbywają się głównie za pomocą komputerów i sieci. Jego zasady działania są kluczowe dla zrozumienia sposobu funkcjonowania systemów kryptograficznych oraz wdrażania skutecznych mechanizmów zabezpieczających przed dostępem osób nieuprawnionych do poufnych danych.

Podstawowym celem szyfrowania jest zapewnienie poufności danych poprzez przekształcenie ich w sposób niezrozumiały dla osób nieupoważnionych. Proces ten tworzy tzw. tekst zaszyfrowany, który jest niemożliwy do zrozumienia bez użycia odpowiedniego klucza deszyfrującego. W konsekwencji, nawet jeśli osoba nieuprawniona uzyska dostęp do zaszyfrowanego tekstu, nie będzie mogła odczytać jego zawartości.

Algorytm szyfrowania to zestaw instrukcji i procedur matematycznych, które określają sposób przekształcenia danych z postaci czytelnej na nieczytelną. Każdy algorytm posiada swoje unikalne cechy i właściwości, które wpływają na jego skuteczność i bezpieczeństwo. Przykłady popularnych algorytmów to AES (Advanced Encryption

Standard), DES (Data Encryption Standard), RSA (Rivest-Shamir-Adleman) oraz SHA (Secure Hash Algorithm).

Klucz natomiast jest parametrem, który wpływa na proces szyfrowania i deszyfrowania danych. W przypadku szyfrowania symetrycznego, ten sam klucz jest używany zarówno do szyfrowania, jak i deszyfrowania danych. Jest to kluczowy aspekt tego rodzaju szyfrowania, ponieważ nadawca i odbiorca muszą mieć dostęp do tego samego klucza. W szyfrowaniu asymetrycznym, używane są pary kluczy: publiczny i prywatny. Klucz publiczny służy do szyfrowania danych, podczas gdy klucz prywatny jest używany do ich deszyfrowania. To podejście umożliwia bezpieczną komunikację między stronami, ponieważ klucz prywatny jest znany tylko odbiorcy.

Podstawowe zasady działania szyfrowania obejmują:

**KonfidenCjonalność**: Szyfrowanie zapewnia poufność danych poprzez utrudnienie dostępu do treści przez osoby nieuprawnione. Tekst zaszyfrowany jest niemożliwy do zrozumienia bez użycia odpowiedniego klucza deszyfrującego.

**Integralność:** Szyfrowanie chroni integralność danych, zapobiegając ich modyfikacji lub manipulacji przez osoby trzecie podczas transmisji lub przechowywania.

**Autentyczność:** Szyfrowanie może również zapewnić autentyczność danych, umożliwiając odbiorcy potwierdzenie, że dane pochodzą od rzeczywistego nadawcy i nie zostały zmienione w trakcie transmisji.

W dzisiejszych czasach, gdzie cyberzagrożenia są coraz bardziej powszechne, szyfrowanie odgrywa kluczową rolę w zapewnianiu bezpieczeństwa danych oraz prywatności użytkowników. Jego zasady działania stanowią fundamenty dla implementacji różnych mechanizmów ochrony danych, zarówno w sferze osobistej, jak i korporacyjnej. Dlatego też, zrozumienie tych zasad oraz stosowanie skutecznych metod szyfrowania jest niezwykle istotne dla zapewnienia
Oprócz podstawowych zasad działania szyfrowania, istnieją również pewne zagadnienia, które warto uwzględnić przy implementacji i stosowaniu tej techniki.

**Wybór odpowiedniego algorytmu:** Istnieje wiele różnych algorytmów szyfrowania, z których każdy ma swoje unikalne cechy i poziom bezpieczeństwa. Ważne jest, aby wybrać algorytm odpowiedni do konkretnego zastosowania,

biorąc pod uwagę jego skuteczność w ochronie danych oraz wydajność obliczeniową.

**Zarządzanie kluczami:** Bezpieczeństwo szyfrowania w dużej mierze zależy od właściwego zarządzania kluczami. Należy zapewnić, aby klucze były odpowiednio długie, losowe i chronione przed nieautoryzowanym dostępem. Ponadto istotne jest regularne rotowanie kluczy oraz wykorzystanie różnych kluczy dla różnych celów (np. klucz szyfrowania i klucz autoryzacji).

**Bezpieczeństwo implementacji:** Sam algorytm szyfrowania może być bezpieczny, ale sposób jego implementacji może prowadzić do luk w bezpieczeństwie. Warto zwrócić uwagę na zabezpieczenie procesów szyfrowania i deszyfrowania, unikanie ataków typu side-channel oraz regularne aktualizacje oprogramowania w celu łatania ewentualnych luk bezpieczeństwa.

**Zarządzanie cyklem życia kluczy:** Klucze kryptograficzne mają swój cykl życia, który obejmuje generowanie, dystrybucję, użycie, rotację i usunięcie. Warto opracować politykę zarządzania cyklem życia kluczy, aby zapewnić ich bezpieczne i skuteczne zarządzanie przez cały ich okres użytkowania.

**Zgodność z przepisami prawnymi i regulacjami:** W niektórych przypadkach, zwłaszcza w sektorach regulowanych, istnieją specjalne przepisy prawne i regulacje dotyczące stosowania szyfrowania danych. Należy upewnić się, że implementacja szyfrowania jest zgodna z obowiązującymi przepisami oraz uwzględnia wymogi dotyczące przechowywania i przetwarzania danych.

**Szyfrowanie na różnych warstwach:** W celu zapewnienia kompleksowego bezpieczeństwa danych, warto rozważyć stosowanie szyfrowania na różnych warstwach infrastruktury IT, takich jak dane przechowywane na dyskach, transmisje danych przez sieci, a także dane przetwarzane w aplikacjach.

Wdrażanie i stosowanie zaawansowanych mechanizmów szyfrowania wymaga zrozumienia nie tylko podstawowych zasad działania tej techniki, ale także uwzględnienia różnych aspektów związanych z bezpieczeństwem danych, zarządzaniem kluczami oraz zgodnością z przepisami prawnymi i regulacjami. Dzięki odpowiedniej implementacji i prawidłowemu zarządzaniu, szyfrowanie może skutecznie zapewnić ochronę danych oraz prywatność użytkowników w dzisiejszym świecie cyfrowym.

# 5. Czym jest i jak działa SSL

**SSL, czyli Secure Sockets Layer**, to protokół bezpieczeństwa stosowany w komunikacji między klientem a serwerem w sieci internetowej. Jego głównym celem jest zabezpieczenie przesyłanych danych poprzez szyfrowanie, co chroni informacje poufne przed potencjalnym przechwyceniem lub manipulacją przez nieupoważnione osoby.

Podstawowym zagrożeniem, przed którym chroni SSL, jest tzw. atak typu "Man-in-the-Middle", gdzie złośliwy użytkownik próbuje przechwycić lub zmodyfikować przesyłane dane pomiędzy użytkownikiem a serwerem. Protokół ten gwarantuje, że nawet jeśli atakujący przechwyci transmisję, nie będzie w stanie odczytać zawartości, ponieważ jest ona zaszyfrowana.

**Główne elementy SSL to:**

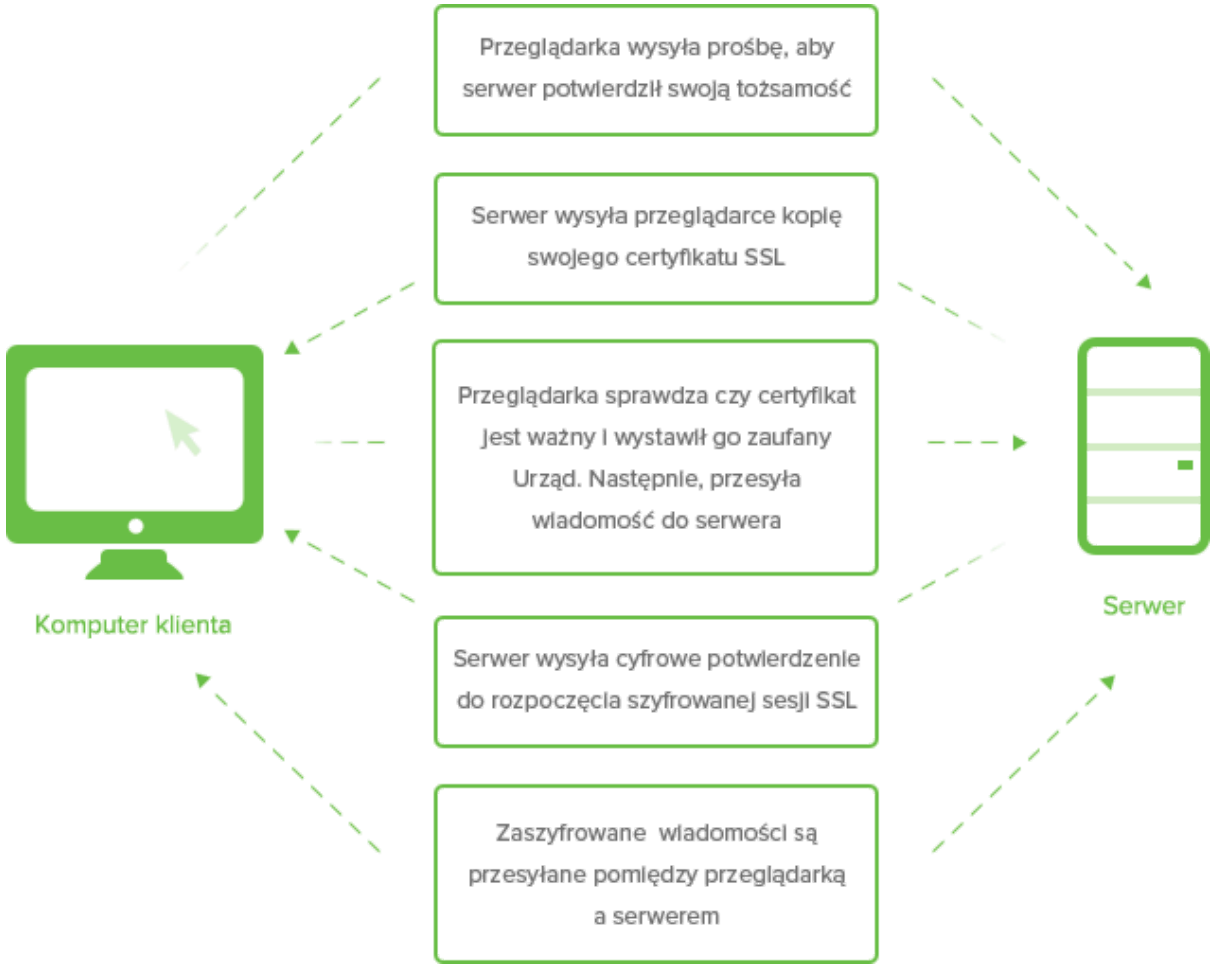
Szyfrowanie danych: SSL wykorzystuje różne algorytmy kryptograficzne do zakodowania informacji przesyłanych między przeglądarką a serwerem. Dzięki temu, nawet jeśli dane zostaną przechwycone, są praktycznie nieczytelne dla osób nieposiadających odpowiednich kluczy.

Autentykacja: Protokół SSL umożliwia również autentykację serwera i/lub klienta. Oznacza to, że strony biorące udział w komunikacji mogą potwierdzić swoją tożsamość. Dzięki temu użytkownicy mają pewność, że łączą się z prawdziwym serwerem, co minimalizuje ryzyko ataków typu "phishing".

Integralność danych: SSL zapewnia integralność danych, co oznacza, że informacje przesyłane między klientem a serwerem nie zostały zmienione podczas transmisji. Jeśli jakiekolwiek zmiany byłyby wprowadzone, protokół wykryje to i uniemożliwi dalszą komunikację.

SSL ewoluował wraz z czasem, a jego rozwinięta wersja to TLS (Transport Layer Security). Terminologia "SSL" jest nadal używana powszechnie, ale technologicznie odnosi się do starszych wersji protokołu. Współcześnie, zaleca się korzystanie z nowszych wersji TLS w celu zapewnienia bezpieczeństwa komunikacji internetowej.

**Jak przebiega transmisja danych z użyciem protokołu SSL?**




**Rodzaje certyfikatów SSL:**

- **Certyfikaty DV:**
  są uznawane za najprostsze i najbardziej podstawowe, ale nadal pełnią istotną rolę w zapewnianiu bezpieczeństwa w Internecie.

  Oto kilka cech charakteryzujących certyfikaty DV:

  Weryfikacja domeny: Certyfikaty DV są emitowane po przejściu procesu weryfikacji, który sprawdza, czy osoba lub organizacja składająca wniosek o certyfikat ma kontrolę nad domeną, dla której certyfikat jest wydawany. Weryfikacja ta może obejmować potwierdzenie poprzez e-mail, dodanie specjalnego rekordu DNS lub umieszczenie pliku na serwerze.

  Niska cena i szybkość wydawania: Certyfikaty DV są zazwyczaj dostępne w niskich cenach, a ich proces weryfikacji jest szybki i prosty. Dzięki temu są

popularnym wyborem dla małych witryn internetowych oraz dla osób, które chcą szybko wdrożyć szyfrowanie na swojej stronie.

Brak szczegółowych informacji o właścicielu: W odróżnieniu od bardziej zaawansowanych certyfikatów, takich jak certyfikaty OV (Organization Validation) i EV (Extended Validation), certyfikaty DV nie zawierają szczegółowych informacji o właścicielu, takich jak nazwa firmy czy miejsce siedziby.

Wsparcie dla protokołów szyfrowania: Mimo że są to podstawowe certyfikaty, zapewniają one silne szyfrowanie dla połączeń HTTPS, co pomaga w ochronie prywatności danych przesyłanych między użytkownikiem a serwerem.

- **Certyfikaty OV:**
  Certyfikaty OV oferują wyższy poziom weryfikacji niż certyfikaty DV (Domain Validation), co sprawia, że są bardziej zaawansowane i wiarygodne.

  Weryfikacja organizacji: Proces weryfikacji przy wydawaniu certyfikatów OV obejmuje potwierdzenie tożsamości i legalności organizacji składającej wniosek. Emitent certyfikatu sprawdza, czy organizacja istnieje, czy jest zarejestrowana i czy ma prawo korzystać z danej domeny.

  Rozszerzone informacje o właścicielu: Certyfikaty OV zawierają rozszerzone informacje o właścicielu, takie jak nazwa firmy, miejsce siedziby oraz informacje kontaktowe. To umożliwia użytkownikom lepsze zweryfikowanie tożsamości strony internetowej, z którą się łączą.

  Większa wiarygodność: Dzięki bardziej zaawansowanej weryfikacji i zawartym informacjom o organizacji certyfikaty OV są zazwyczaj postrzegane jako bardziej wiarygodne niż certyfikaty DV. Są to często wybierane przez firmy i instytucje, które pragną zwiększyć poziom zaufania użytkowników do swojej witryny.

  Wsparcie dla silnego szyfrowania: Certyfikaty OV, podobnie jak certyfikaty DV, oferują silne szyfrowanie dla połączeń HTTPS, co przyczynia się do zabezpieczenia prywatności przesyłanych danych.

  Koszt i czas wydawania: Certyfikaty OV są zazwyczaj droższe niż certyfikaty DV, a proces ich weryfikacji może trwać nieco dłużej ze względu na bardziej szczegółowe sprawdzanie organizacji.

- **Certyfikaty EV:**
  Certyfikaty EV oferują najwyższy poziom weryfikacji i wiarygodności, co sprawia, że są uznawane za najbardziej zaufane wśród wszystkich rodzajów certyfikatów SSL.

  Oto kilka kluczowych cech charakteryzujących certyfikaty EV:

  Rozszerzona weryfikacja: Proces weryfikacji przy wydawaniu certyfikatów EV jest bardzo szczegółowy i obejmuje potwierdzenie tożsamości, prawnego statusu i fizycznej lokalizacji organizacji składającej wniosek. Weryfikowane są także uprawnienia osoby składającej wniosek.

  Rozszerzone informacje o właścicielu: Certyfikaty EV zawierają najbardziej szczegółowe informacje o właścicielu w porównaniu do innych kategorii certyfikatów SSL. Zazwyczaj obejmują nazwę firmy, miejsce siedziby, numer rejestracyjny, a także informacje kontaktowe.

  Zielona belka adresu: Jedną z najbardziej charakterystycznych cech certyfikatów EV jest pojawienie się tzw. "zielonej belki" w pasku adresu przeglądarki. To wizualne wyróżnienie sygnalizuje użytkownikom, że połączenie z witryną jest bezpieczne i zostało zweryfikowane na najwyższym poziomie.

  Najwyższy poziom zaufania: Certyfikaty EV są powszechnie uważane za najbardziej zaufane, co sprawia, że są szczególnie popularne w przypadku stron wymagających najwyższego stopnia bezpieczeństwa, takich jak banki, sklepy internetowe czy inne usługi finansowe.

  Wsparcie dla silnego szyfrowania: Podobnie jak w przypadku innych certyfikatów SSL, certyfikaty EV oferują silne szyfrowanie dla połączeń HTTPS, co jest kluczowe dla ochrony prywatności przesyłanych danych.

  Koszt i czas wydawania: Certyfikaty EV są zazwyczaj najdroższe, a proces weryfikacji może trwać dłużej niż w przypadku innych rodzajów certyfikatów ze względu na skomplikowany proces weryfikacji.

**Przed czym chroni SSL?**
Protokół SSL chroni Cię podczas wizyty na stronie www. Zabezpieczy przed działalnością internetowych przestępców, np.:

- przekierowaniem na inną stronę (phishing – podszycie się pod inną firmę/instytucję w celu kradzieży danych),
- wykradzeniem danych, które podałeś podczas logowania/rejestracji konta,
- przechwyceniem danych użytych przy realizacji e-płatności,
- kradzieżą haseł, numerów kart, kont, a potem użyciem ich do popełnienia przestępstw lub sprzedaży w Darknecie,
- skasowaniem lub fałszowaniem danych,
- kradzieżą danych, które służą do logowania się do sieci wewnętrznych w firmach.

**Darmowy czy płatny certyfikat?**
Różnice między darmowym a płatnym certyfikatem SSL (Secure Sockets Layer) dotyczą głównie zakresu oferowanych funkcji, poziomu weryfikacji oraz wsparcia technicznego. Oto kilka kluczowych różnic między tymi dwoma rodzajami certyfikatów:

- **Poziom weryfikacji tożsamości:**

  Darmowe certyfikaty: Darmowe certyfikaty są zazwyczaj wydawane na podstawie automatycznej weryfikacji, co oznacza, że proces sprawdzania tożsamości właściciela strony jest mniej rygorystyczny. To może oznaczać mniejsze zaufanie, gdyż certyfikat nie potwierdza tożsamości firmy czy osoby z taką dokładnością, jak płatne certyfikaty.

  Płatne certyfikaty: Płatne certyfikaty wymagają bardziej rygorystycznej procedury weryfikacji tożsamości. Producenci płatnych certyfikatów (Certificate Authorities - CA) przeprowadzają staranne badania, aby potwierdzić, czy osoba lub firma składająca wniosek o certyfikat jest właścicielem domeny.

- **Zakres gwarancji:**

  Darmowe certyfikaty: W przypadku darmowych certyfikatów, gwarancje finansowe zazwyczaj są niewielkie lub nawet brak ich w ogóle. Jeśli więc użytkownik strony padnie ofiarą ataku związanej z certyfikatem SSL, może być trudno uzyskać odszkodowanie.

  Płatne certyfikaty: Płatne certyfikaty zazwyczaj oferują pewien poziom gwarancji finansowej. Oznacza to, że w przypadku problemów związanych z błędami w certyfikacie, użytkownik może mieć możliwość otrzymania rekompensaty finansowej.

- **Wsparcie techniczne:**

  Darmowe certyfikaty: W przypadku darmowych certyfikatów, wsparcie techniczne może być ograniczone lub wcale nie istnieć. Oferowane narzędzia i dokumentacja mogą być jedynymi dostępnymi źródłami pomocy.

  Płatne certyfikaty: Płatne certyfikaty zazwyczaj oferują lepsze wsparcie techniczne. Firmy oferujące płatne certyfikaty mogą udzielać pomocy w implementacji, rozwiązywaniu problemów związanych z certyfikatem, a także w kwestiach związanych z bezpieczeństwem.

- **Ważność certyfikatu:**

  Darmowe certyfikaty: Często darmowe certyfikaty mają krótszy okres ważności w porównaniu do płatnych, co oznacza, że wymagają częstszej aktualizacji.

  Płatne certyfikaty: Płatne certyfikaty zazwyczaj oferują dłuższe okresy ważności, co może być wygodne dla administratorów systemów.

# 6. Czym jest i jak działa TLS

**Transport Layer Security (TLS)** to protokół bezpieczeństwa, który, podobnie jak SSL (Secure Sockets Layer), służy do zabezpieczania komunikacji internetowej między klientem a serwerem. Jego głównym celem jest ochrona poufności, integralności i autentykacji przesyłanych danych.

Szyfrowanie danych: Podobnie jak SSL, TLS wykorzystuje różne algorytmy kryptograficzne do szyfrowania danych przesyłanych między przeglądarką a serwerem. Ten proces sprawia, że nawet jeśli dane są przechwycone, są praktycznie nieczytelne dla osób nieupoważnionych, ponieważ wymagają odpowiednich kluczy do odszyfrowania.

Autentykacja: TLS umożliwia autentykację zarówno serwera, jak i klienta. Dzięki temu strony komunikujące się ze sobą mogą zweryfikować swoją tożsamość. To zabezpiecza użytkowników przed potencjalnymi atakami typu "phishing", zapewniając, że łączą się z prawdziwymi serwerami.

Integralność danych: Protokół TLS gwarantuje integralność danych, co oznacza, że przesyłane informacje nie mogą być modyfikowane podczas transmisji. Jeśli

jakiekolwiek zmiany zostaną wprowadzone, protokół to wykryje, uniemożliwiając dalszą komunikację.

Rozszerzalność: TLS jest protokołem rozwiniętym na bazie SSL. Posiada szereg ulepszeń i poprawek, co czyni go bardziej bezpiecznym. Ponadto, TLS jest bardziej elastyczny i zdolny do obsługi różnych algorytmów szyfrowania oraz innych funkcji bezpieczeństwa.

Wsparcie dla starszych wersji SSL: W celu zachowania kompatybilności, TLS często obsługuje starsze wersje SSL, ale zaleca się korzystanie z nowszych wersji protokołu ze względów bezpieczeństwa.

TLS jest obecnie bardziej powszechnie stosowanym protokołem niż SSL, a używanie najnowszych wersji TLS jest zalecane, aby utrzymać wysoki poziom bezpieczeństwa w komunikacji internetowej.

**Wersje protokołu TLS:**

- **TLS 1.0:**
  Jest to pierwotna wersja protokołu TLS, a wcześniej był znany jako SSL 3.1 (Secure Sockets Layer).
  Został wprowadzony w 1999 roku jako następca protokołu SSL 3.0.
  Mimo że zapewniał podstawową ochronę przed niektórymi atakami, z czasem uznano go za podatny na pewne zagrożenia, w tym na tzw. ataki typu "BEAST" i "POODLE".
  Zaleca się unikanie użycia TLS 1.0 ze względu na jego przestarzałość i słabe bezpieczeństwo.

- **TLS 1.1:**
  Wprowadzony jako poprawka do TLS 1.0 w 2006 roku, w celu naprawienia pewnych luk w zabezpieczeniach poprzedniej wersji.
  Zastosowano kilka poprawek, w tym m.in. do obsługi algorytmów szyfrowania.
  Pomimo prób ulepszenia bezpieczeństwa, TLS 1.1 również został uznany za przestarzały ze względu na istnienie bardziej zaawansowanych wersji.

- **TLS 1.2:**
  Jest obecnie szeroko stosowaną wersją protokołu TLS.
  Wprowadza liczne usprawnienia w stosunku do poprzednich wersji, takie jak zwiększenie liczby obsługiwanych algorytmów szyfrowania, poprawy w zakresie bezpieczeństwa i skuteczności negocjacji.

Zapewnia również obsługę zaawansowanych funkcji, takich jak tzw. Perfect Forward Secrecy (PFS), co oznacza, że nawet jeśli klucz zostanie skompromitowany, poprzednie komunikacje nie zostaną naruszone.

- **TLS 1.3:**
  Jest najnowszą wersją protokołu TLS, wprowadzoną w 2018 roku.
  Zmniejsza czas negocjacji i poprawia ogólną wydajność w porównaniu do TLS 1.2.
  Usuwa niektóre starsze i mniej bezpieczne algorytmy, skupiając się na bardziej nowoczesnych i bezpiecznych rozwiązaniach.
  Wprowadza również funkcje, takie jak 0-RTT (Zero Round Trip Time Resumption), które mają na celu jeszcze bardziej przyspieszyć proces nawiązywania połączenia.

**Jak przebiega transmisja danych z użyciem protokołu TLS?**

Proces transmisji danych z użyciem protokołu TLS (Transport Layer Security) obejmuje kilka kroków, a cały proces jest zaprojektowany w celu zapewnienia bezpiecznej komunikacji pomiędzy klientem a serwerem.

- **Inicjacja połączenia (Handshake):**
  Klient wysyła żądanie połączenia: Klient inicjuje komunikację z serwerem, wysyłając żądanie nawiązania połączenia TLS.
  Wstępny przegląd algorytmów: Klient i serwer wymieniają się informacjami dotyczącymi obsługiwanych algorytmów szyfrowania, kluczy i innych ustawień zabezpieczeń.

- **Wymiana kluczy (Key Exchange):**
  Ustalenie klucza sesji: Klient i serwer ustalają wspólny sekret (klucz sesji) za pomocą algorytmu wymiany kluczy, na przykład Diffie-Hellman.

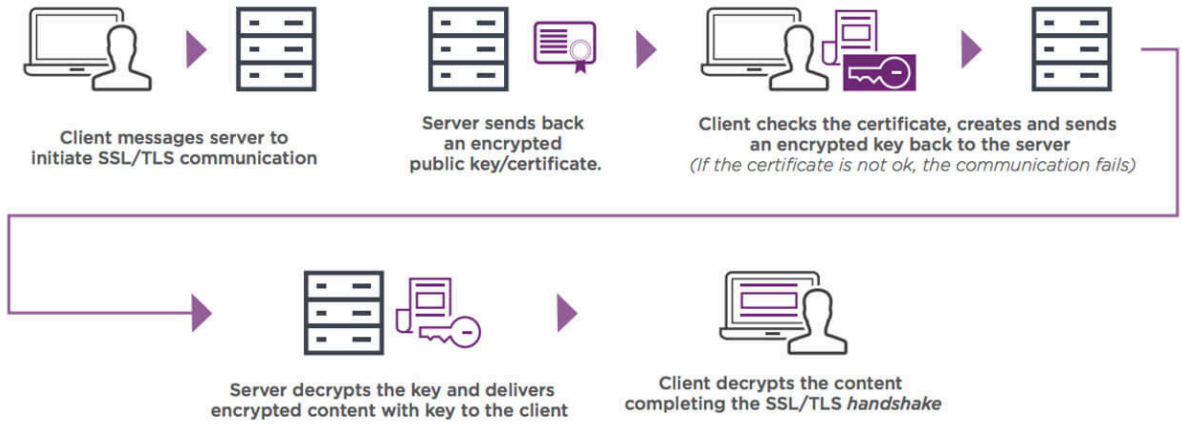
- **Weryfikacja tożsamości (Authentication):**
  Certyfikat serwera: Serwer prezentuje swój certyfikat cyfrowy potwierdzający tożsamość. Klient sprawdza, czy certyfikat jest zaufany i prawidłowy.
  Opcjonalna weryfikacja klienta: W niektórych przypadkach klient może zostać poproszony o przedstawienie swojego certyfikatu w celu zweryfikowania jego tożsamości.

- **Ustalenie parametrów sesji (Session Parameters):**

Ustalenie parametrów szyfrowania: Klient i serwer uzgadniają szczegóły dotyczące używanych algorytmów szyfrowania, metod kompresji, itp.

- **Utworzenie tunelu bezpieczeństwa (Security Tunnel):**
  Rozpoczęcie bezpiecznej transmisji: Po zakończeniu handshake, klient i serwer korzystają z ustalonego klucza sesji do szyfrowania danych przesyłanych między nimi.
  Rozpoczęcie transmisji danych: Po nawiązaniu bezpiecznego połączenia, klient i serwer mogą bezpiecznie przesyłać dane między sobą przez tunel bezpieczeństwa.

Warto zauważyć, że protokół TLS jest zazwyczaj zaimplementowany na warstwie transportowej modelu OSI, tj. nad warstwą transportową (np. TCP). Dzięki temu zapewnia bezpieczną transmisję danych w kontekście protokołów komunikacyjnych, takich jak HTTP (w przypadku HTTPS) czy też SMTP (w przypadku STARTTLS).




**Przyszłość TLS:**

Przyszłość bezpieczeństwa w Internecie, w tym protokołu Transport Layer Security (TLS), obejmuje kilka kluczowych obszarów rozwoju i zmian. Poniżej przedstawiam kilka istotnych kierunków, które mogą wpłynąć na przyszłość TLS i ogólnie bezpieczeństwa w sieci:

Rozwój protokołów TLS: Nowe wersje protokołu TLS są tworzone w celu poprawy bezpieczeństwa i wydajności. TLS 1.3, wprowadzony w 2018 roku, znacząco poprawił wiele aspektów, takie jak szybkość nawiązywania połączeń i eliminacja niebezpiecznych algorytmów szyfrowania. W przyszłości mogą pojawić się kolejne wersje z dodatkowymi usprawnieniami.

Kwantowe zagrożenia: Rozwój komputerów kwantowych może stanowić wyzwanie dla aktualnych algorytmów szyfrowania wykorzystywanych w TLS. Pracuje się nad algorytmami kwantoodpornymi, które mają być odporne na ataki kwantowe. Zabezpieczenia przed tymi zagrożeniami mogą być konieczne w przyszłości.

Automatyzacja i zarządzanie certyfikatami: Wzrastające znaczenie zarządzania certyfikatami SSL/TLS stawia wyzwanie przed organizacjami. Rozwinięcie automatyzacji procesów uzyskiwania, odnawiania i zarządzania certyfikatami może poprawić efektywność i zwiększyć bezpieczeństwo.

Większe zastosowanie szyfrowania: W miarę rozwoju technologii oczekuje się, że coraz więcej danych będzie przesyłanych zaszyfrowanymi kanałami. To obejmuje zarówno ruch sieciowy, jak i komunikację między aplikacjami. Wzrost szyfrowania wymaga równoczesnego rozwoju silnych i wydajnych protokołów bezpieczeństwa, takich jak TLS.

TLS w protokołach aplikacyjnych: Użycie TLS nie ogranicza się już tylko do przeglądarek internetowych. Wzrasta zastosowanie TLS w protokołach aplikacyjnych, takich jak gry online, aplikacje mobilne, komunikatory, itp. Dalszy rozwój i dostosowanie TLS do różnych kontekstów aplikacyjnych są kluczowe.

Ochrona prywatności i standardy: Znaczenie ochrony prywatności staje się coraz bardziej centralne. Wprowadzenie nowych standardów i inicjatyw, takich jak Private Internet Access (PIA) czy standardy Privacy Pass, może wpłynąć na przyszłość bezpieczeństwa w Internecie.

Ataki i wyzwania: Ataki na protokoły bezpieczeństwa zawsze będą stanowiły wyzwanie. W miarę jak cyberprzestępcy rozwijają nowe metody ataków, konieczne jest ciągłe dostosowywanie i ulepszanie protokołów bezpieczeństwa.

Przyszłość TLS i bezpieczeństwa w Internecie będzie zależała od dynamicznego rozwoju technologii oraz zdolności do skutecznego reagowania na nowe zagrożenia. W miarę postępujących zmian w środowisku cybernetycznym, kluczowe będzie utrzymywanie aktualizacji i stosowanie najnowszych praktyk bezpieczeństwa.

**Częste błędy przy wprowadzaniu protokołu TLS:**

Protokół Transport Layer Security (TLS) jest powszechnie używany do zabezpieczania komunikacji w sieciach, szczególnie w przypadku przeglądania stron internetowych,

poczty elektronicznej, czy transmisji danych. Jednak istnieją pewne częste błędy i wyzwania związane z TLS, które mogą prowadzić do problemów z bezpieczeństwem lub dostępnością. Oto kilka z nich:

- Ustępujące wersje TLS/SSL:
  Nieaktualizowane wersje TLS/SSL, takie jak TLS 1.0 czy SSLv3, są podatne na różne ataki. Ważne jest, aby serwery i aplikacje korzystały z najnowszych wersji protokołu TLS.

- Niewłaściwe konfiguracje czołówki bezpieczeństwa:
  Niewłaściwe konfiguracje, takie jak słabe algorytmy szyfrowania, zbyt krótkie klucze, czy nie aktualne certyfikaty mogą osłabić bezpieczeństwo komunikacji. Zaleca się korzystanie z silnych algorytmów i regularne aktualizacje certyfikatów.

- Problem z certyfikatem:
  Niewłaściwie skonfigurowane certyfikaty SSL/TLS mogą prowadzić do ostrzeżeń lub blokad przeglądarek. Upewnij się, że certyfikaty są ważne, poprawnie podpisane i skonfigurowane zgodnie z najlepszymi praktykami.

- Ataki typu Man-in-the-Middle (MitM):
  Ataki tego typu polegają na podszywaniu się pod prawdziwe strony lub serwery, co może prowadzić do przechwytywania poufnych informacji. Dlatego ważne jest, aby używać certyfikatów SSL/TLS od zaufanych dostawców i monitorować ruch w poszukiwaniu podejrzanej aktywności.

- Zły zarząd certyfikatami:
  Brak aktualizacji certyfikatów, nieprawidłowa konfiguracja ich odnowień i brak monitorowania ich stanu może prowadzić do problemów z dostępem do zasobów zabezpieczonych protokołem TLS.

- Słabe konfiguracje serwera:

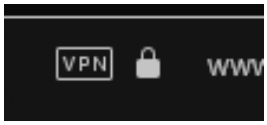
  Niewłaściwe ustawienia serwera, takie jak niedostateczna konfiguracja nagłówków bezpieczeństwa, brak obsługi HSTS (HTTP Strict Transport Security) czy brak konfiguracji OCSP (Online Certificate Status Protocol), mogą wpłynąć na bezpieczeństwo komunikacji.

- Problemy z negocjacją TLS:
  W niektórych przypadkach może dojść do problemów z negocjacją parametrów TLS pomiędzy klientem a serwerem, co może prowadzić do błędów komunikacji.

Ważne jest, aby dostosować konfigurację po obu stronach w celu zapewnienia zgodności.

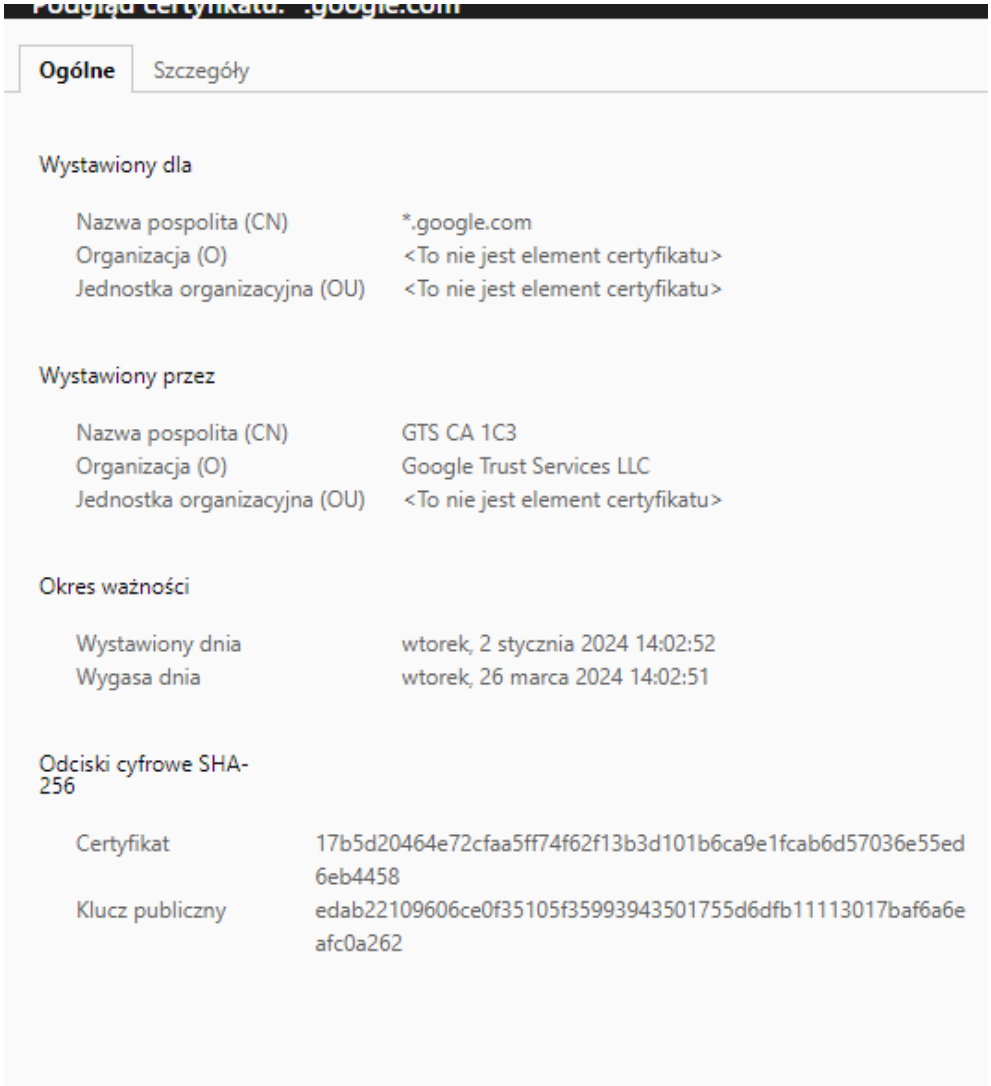
**Po czym rozpoznać, że strona korzysta z certyfikatu bezpieczeństwa?**

W przypadku stron korzystających z szyfrowania, w pasku adresu znajduje się przedrostek HTTPS://. z angielskiego Hypertext Transfer Protocol Secure.




Symbol zamkniętej kłódki, jest to kolejne oznaczenie zaraz obok pasku adresu, które informuje nas o szyfrowaniu strony. Po kliknięciu w kłódkę, otrzymujemy informację o ważności certyfikatu.

Możemy również uzyskać szczegółowe informację na temat certyfikatu na każdej stronie, przykład ze stronny google poniżej:




**Jak odróżnić stronę bez protokołu bezpieczeństwa?**
W przypadku stron, które nie korzystają z szyfrowania, przy pasku adresu pojawi się odpowiedni komunikat o braku zabezpieczenia. Podczas tworzenia własnej strony warto zadbać o to by posiadała ona odpowiedni protokół, gdyż informacja o braku zabezpieczenia odstrasza potencjalnych odwiedzających już na samym starcie.

**Jakie korzyści niesie za sobą korzystanie z protokołów bezpieczeństwa?**
Stosowanie certyfikatu SSL (Secure Sockets Layer) lub jego bardziej nowoczesnej wersji TLS (Transport Layer Security) niesie ze sobą kilka kluczowych korzyści:

- Poufność danych:
  Certyfikaty SSL/TLS zapewniają szyfrowanie danych między klientem a serwerem. To oznacza, że nawet jeśli dane są przechwycone przez nieupoważnioną osobę lub program, nie mogą być odczytane, ponieważ są zakodowane.

- Integralność danych:
  Certyfikaty SSL zapewniają integralność przesyłanych danych. Dzięki mechanizmom kontroli integralności, gwarantują, że dane nie zostały zmienione lub sfałszowane w trakcie przesyłania.

- Uwierzytelnianie serwera:
  Certyfikaty SSL umożliwiają uwierzytelnianie serwera w trakcie nawiązywania połączenia. To potwierdzenie tożsamości serwera pomaga zapobiegać atakom typu "man-in-the-middle", w których atakujący próbuje podszyć się za serwer.

- Zaufanie użytkowników:
  Strony internetowe z certyfikatem SSL są oznaczone jako "bezpieczne" w przeglądarkach internetowych. To oznaczenie buduje zaufanie użytkowników do witryny, co może zwiększyć konwersję i zachęcać do bezpiecznego korzystania z usług.

- Zgodność z regulacjami:
  W niektórych branżach istnieją regulacje, które wymagają stosowania certyfikatów SSL/TLS, zwłaszcza tam, gdzie przetwarzane są wrażliwe dane osobowe. Przykładem może być regulacja PCI DSS dla firm obsługujących transakcje kart płatniczych.

- SEO:
  Wprowadzenie SSL/TLS może mieć wpływ na wyniki wyszukiwania w Google. W 2014 roku Google ogłosiło, że strony z certyfikatem SSL mogą uzyskać pewną przewagę w wynikach wyszukiwania w porównaniu do niezabezpieczonych stron.

- Zabezpieczenie przed atakami:
  Certyfikaty SSL/TLS pomagają w zabezpieczeniu przed różnymi rodzajami ataków, takimi jak przechwytywanie danych, ataki typu "man-in-the-middle", czy ataki związane z podszywaniem się za serwer.

- Zwiększenie zaufania do marki:

Posiadanie certyfikatu SSL świadczy o trosce o bezpieczeństwo użytkowników. To może wpływać pozytywnie na reputację marki i budować zaufanie klientów.

**Czy moc szyfrowania ma znaczenie?**
Moc szyfrowania odgrywa kluczową rolę w zapewnianiu bezpieczeństwa danych w transmisji, szczególnie w kontekście komunikacji internetowej. Szyfrowanie umożliwia zabezpieczenie informacji przed nieuprawnionym dostępem i manipulacją podczas przesyłania ich przez sieć. W kontekście protokołów takich jak SSL (Secure Sockets Layer) i TLS (Transport Layer Security), moc szyfrowania jest kluczowym elementem ochrony danych.

Oto kilka algorytmów szyfrowania używanych w kontekście SSL/TLS:

RSA (Rivest-Shamir-Adleman): RSA jest asymetrycznym algorytmem, co oznacza, że używa dwóch kluczy: publicznego do szyfrowania i prywatnego do deszyfrowania. Często jest używany do wymiany kluczy sesji w celu ustalenia wspólnego klucza dla symetrycznego szyfrowania.

Diffie-Hellman (DHE i ECDHE): To protokół wymiany kluczy, który umożliwia dwóm stronom bezpieczną wymianę kluczy, które są następnie używane do ustalenia wspólnego sekretu do szyfrowania komunikacji.

AES (Advanced Encryption Standard): AES jest symetrycznym algorytmem szyfrowania, co oznacza, że ten sam klucz jest używany do zarówno szyfrowania, jak i deszyfrowania danych. Popularne długości klucza to 128, 192 i 256 bitów.

ChaCha20-Poly1305: Jest to lekki, szybki i bezpieczny algorytm szyfrowania stosowany w protokole TLS.

3DES (Triple DES): Chociaż jest starszy i mniej zalecany ze względu na swoją niższą wydajność w porównaniu do AES, 3DES nadal jest czasami używany w starszych systemach.

Camellia: Jest to symetryczny algorytm szyfrowania, który jest dostępny w niektórych implementacjach TLS.

Ważne jest, aby korzystać z silnych i bezpiecznych algorytmów szyfrowania, ponieważ starsze i mniej bezpieczne algorytmy mogą być podatne na ataki kryptoanalityczne. W miarę postępu technologii zaleca się stosowanie nowoczesnych i bezpiecznych algorytmów szyfrowania.

# 7. Różnice pomiędzy SSL a TLS

W praktyce terminy SSL (Secure Sockets Layer) i TLS (Transport Layer Security) często są używane zamiennie, ale istnieją różnicę między nimi.

SSL to starsza wersja protokołu, wprowadzona przez firmę Netscape, podczas gdy TLS to następca, który został opracowany w celu poprawy bezpieczeństwa i funkcjonalności SSL.

Początkowo, SSL był powszechnie używany do zabezpieczania połączeń internetowych, jednak z czasem zidentyfikowano pewne luki w zabezpieczeniach w starszych wersjach SSL (takie jak SSL 2.0 i SSL 3.0). W związku z tym, zaleca się unikanie starszych wersji SSL na rzecz bardziej bezpiecznych implementacji, takich jak TLS.

Współczesne przeglądarki internetowe i serwery zazwyczaj używają TLS do zapewnienia bezpiecznej komunikacji przez Internet. Kiedy ludzie mówią o korzystaniu z SSL, często mają na myśli ogólnie używanie bezpiecznego protokołu do transmisji danych, a niekoniecznie konkretnej wersji SSL. W praktyce, kiedy mówimy o "HTTPS" (Hypertext Transfer Protocol Secure), często używamy TLS, ponieważ współczesne implementacje HTTPS opierają się głównie na TLS.

Oto kilka różnic między TLS a SSL:

- Nazwa:
  SSL to starsza wersja protokołu, wprowadzona przez firmę Netscape.
  TLS to następca SSL i jest uważany za jego rozwinięcie.

- Wersje:
  SSL miał kilka wersji, takie jak SSL 2.0, SSL 3.0.
  TLS ma swoje wersje, takie jak TLS 1.0, TLS 1.1, TLS 1.2, TLS 1.3 (najnowsza wersja w momencie mojej wiedzy w styczniu 2022).

- Bezpieczeństwo:
  W przypadku bezpieczeństwa, TLS jest zazwyczaj uważane za bardziej bezpieczne niż starsze wersje SSL. W rzeczywistości zaleca się unikanie starszych wersji SSL ze względu na znane luki w zabezpieczeniach.
  Silniejsze Algorytmy Szyfrowania:

  Jednym z głównych czynników poprawiających bezpieczeństwo TLS jest obsługa silniejszych i bardziej zaawansowanych algorytmów szyfrowania w porównaniu do starszych wersji SSL. TLS pozwala na korzystanie z nowoczesnych algorytmów, co sprawia, że transmisja danych jest bardziej odporna na ataki kryptoanalityczne.
  Poprawki Bezpieczeństwa:

  TLS jest aktywnie rozwijane, a jego nowsze wersje wprowadzają poprawki bezpieczeństwa w odpowiedzi na pojawiające się zagrożenia. Dzięki temu można szybko reagować na nowe metody ataków i dostosowywać protokół do zmieniającego się środowiska bezpieczeństwa.
  Unikanie Znanych Ataków:

  W przeszłości starsze wersje SSL były podatne na pewne poważne ataki, takie jak "POODLE" czy "Heartbleed". TLS zostało zaprojektowane z myślą o wyeliminowaniu tych luk bezpieczeństwa, co sprawia, że jest bardziej odporne na tego rodzaju ataki.
  Ochrona Przed Atakami Man-in-the-Middle (MitM):

  TLS wdrożyło skuteczne mechanizmy zapobiegające atakom typu MitM. Protokół ten stosuje proces handshake, który obejmuje negocjację kluczy kryptograficznych między klientem a serwerem, co utrudnia potencjalnym atakującym przechwytywanie i manipulowanie danymi w trakcie transmisji.

Wspieranie Protokołów Bezpieczeństwa Warstwy Aplikacji (TLS 1.3):

Wersja TLS 1.3 wprowadziła kilka kluczowych usprawnień w zakresie bezpieczeństwa, takich jak usunięcie niebezpiecznych funkcji, przyspieszenie procesu handshake i eliminacja niektórych potencjalnych zagrożeń.
Zakazanie Słabych Szyfrów:

TLS pozwala na zakazanie użycia słabych i niebezpiecznych algorytmów szyfrowania, co zwiększa ogólne bezpieczeństwo komunikacji. Starsze wersje SSL mogły być podatne na ataki, które wykorzystywały słabe metody szyfrowania.

- Algorytmy szyfrowania:
  TLS obsługuje szerszy zakres algorytmów szyfrowania w porównaniu do SSL, co pozwala na lepszą bezpieczeństwo i elastyczność w dostosowywaniu do ewentualnych zmian w standardach kryptograficznych.

- Działanie wsteczne:
  W przeciwieństwie do niektórych starszych wersji SSL, które są bardziej podatne na ataki, współczesne implementacje TLS są zazwyczaj bardziej odporne na różne ataki kryptograficzne.

- Handshake:
  Protokół handshake w TLS jest bardziej rozbudowany i elastyczny niż w SSL. TLS handshake pozwala na negocjację bardziej zaawansowanych opcji i algorytmów bezpieczeństwa.

- Poprawki i Aktualizacje:
  W miarę pojawiania się nowych zagrożeń i postępów w dziedzinie kryptografii, TLS jest aktywnie rozwijane i aktualizowane, co pozwala na utrzymanie wysokiego poziomu bezpieczeństwa. SSL, ze względu na swoje starsze wersje, nie jest już aktywnie rozwijane i rekomenduje się unikanie ich w implementacjach.

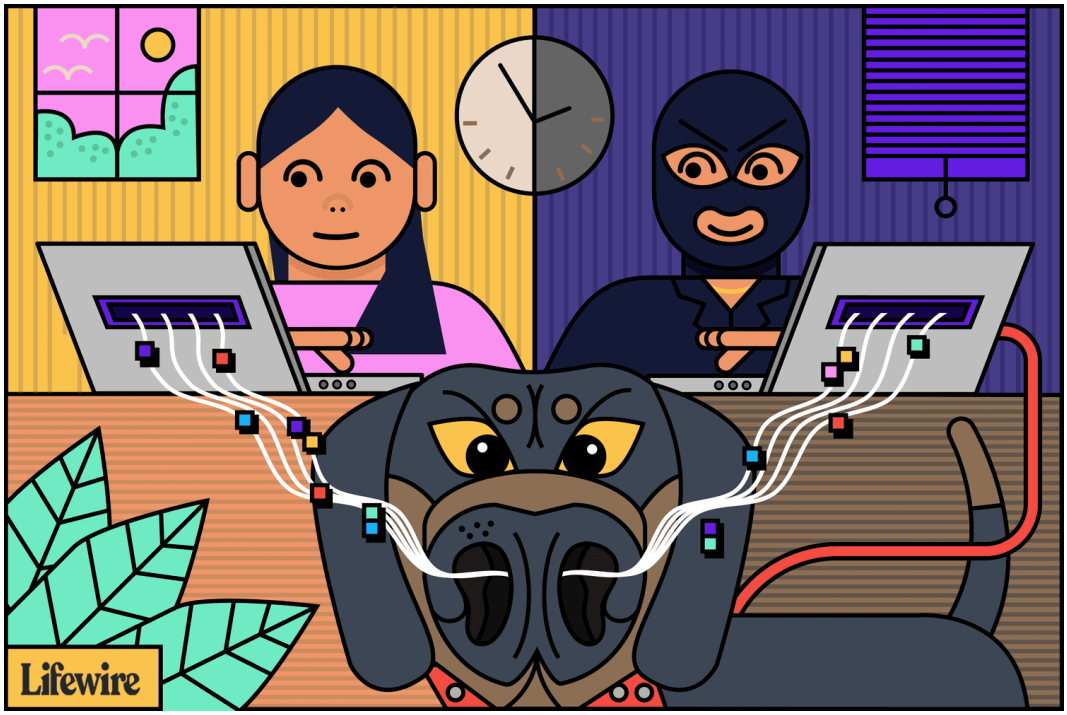
# 8. Metody nasłuchiwania, programy

**Różne metody ataków na informacje:**

- **Sniffing:**

Sniffing, czyli podsłuch sieciowy, to technika używana w dziedzinie informatyki do przechwytywania i analizy pakietów danych przesyłanych przez sieć komputerową. Ten rodzaj aktywności jest zazwyczaj wykonywany w celu monitorowania ruchu sieciowego, analizy komunikacji lub, niestety, także w celu nieuprawnionego dostępu do poufnych informacji.

Sniffing może być zarówno legalny, gdy jest wykonywany w celu diagnostyki i zarządzania siecią, jak i nielegalny, gdy jest używany do podsłuchiwania danych bez zgody właściciela sieci. W przypadku zastosowań legalnych, narzędzia do sniffingu są często używane przez administratorów sieci w celu rozwiązywania problemów z wydajnością, diagnozowania błędów lub analizy ruchu w celu poprawy bezpieczeństwa sieci.

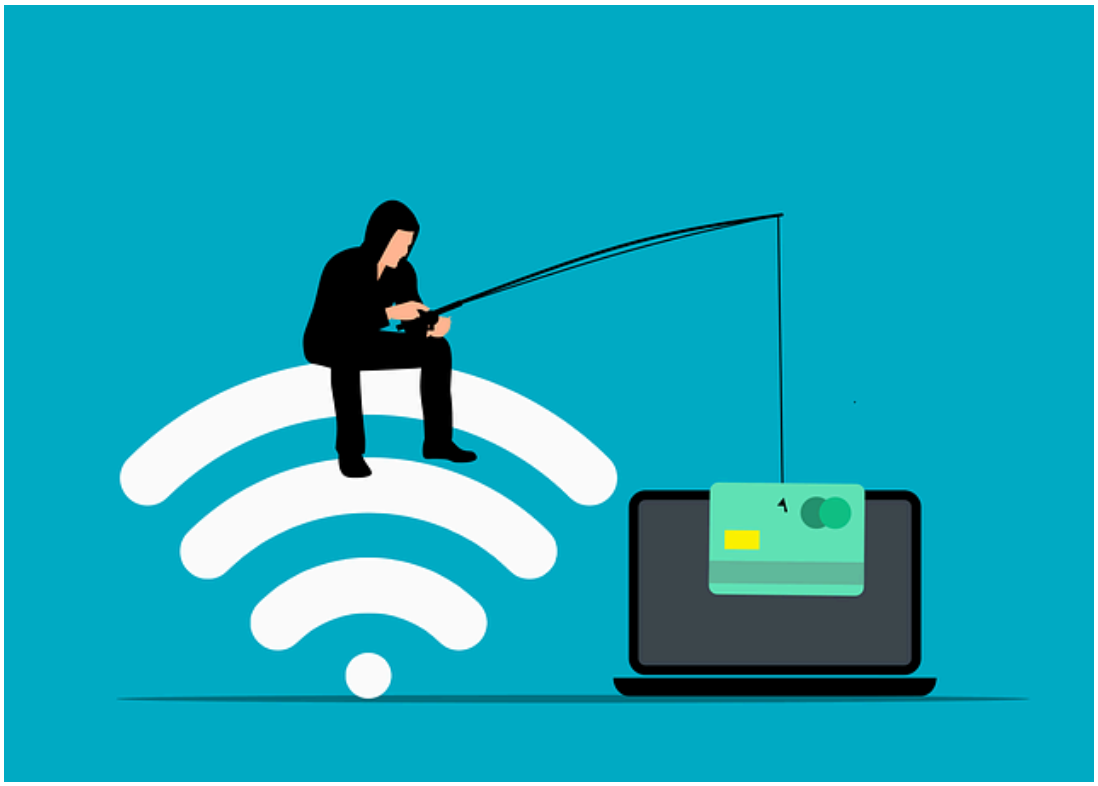



Jednak w kontekście bezpieczeństwa informacyjnego i prywatności, sniffing może być również wykorzystywany przez hakerów do kradzieży poufnych informacji, takich jak hasła, dane osobowe czy inne poufne dane przesyłane przez sieć.

- **Spoofing:**
  Spoofing to technika polegająca na fałszowaniu lub podszywaniu się pod określone dane, adresy lub tożsamość w celu wprowadzenia w błąd lub oszukania systemu, urządzenia lub użytkownika. Istnieje kilka rodzajów spoofingu, z których każdy ma różne zastosowania i cel.

IP Spoofing (Spoofing adresu IP): W tej technice atakujący fałszuje adres IP, aby ukryć prawdziwe źródło pakietów danych. Jest to często wykorzystywane w celu ukrycia tożsamości napastnika podczas ataków DDoS (rozproszonych ataków odmowy usługi).

E-mail Spoofing (Spoofing poczty elektronicznej): W tym przypadku atakujący fałszuje adres nadawcy w wiadomości e-mail w celu wprowadzenia odbiorcy w błąd. Może to być wykorzystywane do prowadzenia ataków phishingowych, gdzie atakujący próbuje pozyskać poufne informacje od odbiorcy, podszywając się pod zaufane źródło.



DNS Spoofing (Spoofing DNS): Atakujący manipuluje rekordami DNS, aby przekierować użytkownika na fałszywe strony internetowe. To może prowadzić do przechwytywania poufnych informacji, takich jak hasła czy dane logowania.

MAC Spoofing (Spoofing adresu MAC): Atakujący zmienia fizyczny adres MAC karty sieciowej, co pozwala mu na podszywanie się pod innego użytkownika w sieci lokalnej.

Caller ID Spoofing (Spoofing numeru telefonu): Atakujący fałszuje numer telefonu wyświetlany na urządzeniu odbiorcy, co może prowadzić do oszustw telefonicznych.

- **Skanowanie portów:**
  Skanowanie portów to proces badania lub sprawdzania, które porty komunikacyjne na danym systemie są otwarte i reagują na żądania. Porty są numerowane i służą jako punkty dostępu do aplikacji lub usług na urządzeniach w sieci. Skanowanie portów może być używane zarówno do celów diagnostycznych, jak i w celach związanych z bezpieczeństwem.

  Istnieje kilka typów skanowania portów, a niektóre z najczęściej używanych to:

  Skanowanie pełnego zakresu (Full Connect Scan): Skanowanie, które próbuje nawiązać połączenie z każdym portem, aby sprawdzić, czy jest on otwarty. To podejście jest dokładne, ale jednocześnie jest bardziej zauważalne dla systemów obronnych.

  Skanowanie SYN (Half-Open Scan lub Stealth Scan): Skanowanie, które wysyła jedynie pakiet SYN (rozpoczęcie połączenia) do celu. Jeśli serwer odpowie pakietem SYN-ACK, oznacza to, że port jest otwarty. Atakujący jednak nie kończy procesu nawiązywania połączenia, co może utrudnić detekcję tego skanowania.

  Skanowanie UDP: UDP (User Datagram Protocol) to protokół bezpołączeniowy, więc skanowanie tego rodzaju jest trudniejsze niż w przypadku TCP. Skanowanie UDP polega na wysyłaniu pakietów do portów UDP i obserwowaniu odpowiedzi. Brak odpowiedzi sugeruje, że port jest zamknięty, ale brak pewności co do tego, czy jest otwarty.

  Skanowanie z użyciem narzędzi automatycznych: Istnieje wiele narzędzi do skanowania portów, takich jak Nmap, które oferują różne opcje i techniki skanowania. Te narzędzia mogą być używane zarówno do legalnego testowania bezpieczeństwa, jak i do nieuprawnionego sondowania systemów w celach związanych z atakami.

- **Phishing:**
  Phishing to złożona i zorganizowana forma cyberprzestępstwa, w ramach której cyberprzestępcy stosują różnorodne strategie i techniki, aby uzyskać nieuprawniony dostęp do poufnych informacji od swoich ofiar. Ataki phishingowe

są często precyzyjnie zaplanowane i wykorzystują psychologiczne sztuczki, aby manipulować ludzkim zachowaniem.

Inżynieria Społeczna: Phishing opiera się na inżynierii społecznej, czyli manipulacji ludzkich emocji i działań. Atakujący starają się stworzyć sytuacje, w których ofiara będzie bardziej skłonna ufać fałszywym źródłom, np. fałszywym e-mailom, wiadomościom tekstowym, czy rozmowom telefonicznym.

Fałszywe Reprezentacje: Ataki phishingowe często polegają na udawaniu instytucji, firm, czy znanych osób. Cyberprzestępcy tworzą fałszywe strony internetowe, fałszywe e-maile lub nawet fałszywe profile społecznościowe, aby zmylić ofiarę co do rzeczywistej tożsamości nadawcy.

Skrupulatna Emulacja: Phishing często polega na skrupulatnej emulacji prawdziwych komunikatów, stron internetowych i wiadomości. E-maile, strony logowania i komunikaty są projektowane w taki sposób, aby wyglądały jak autentyczne, co utrudnia ofiarom rozróżnienie oszukańczych informacji od rzeczywistych.

Dynamiczne Filtry: Atakujący stosują dynamiczne filtry, dostosowując swoje ataki do bieżących wydarzeń, trendów i sytuacji społeczno-gospodarczych. Na przykład, w okresie pandemii, pojawiały się ataki phishingowe związane z tematem COVID-19.

Zaawansowane Techniki: Phishing może wykorzystywać zaawansowane techniki, takie jak pharming, gdzie przechwytywane są ruchy internetowe, aby przekierować ofiary na fałszywe strony. Ponadto, atakujący mogą stosować malware w celu skompromitowania systemów ofiar.

Różnorodność Platform: Atak phishingowy może być przeprowadzany na różnych platformach, w tym e-mail, SMS, media społecznościowe, czy nawet przez rozmowy telefoniczne. Cyberprzestępcy wykorzystują różne środki komunikacji, aby zwiększyć szanse na skuteczność ataku.

**Program WireShark:**
Wireshark to narzędzie do analizy ruchu sieciowego (network protocol analyzer) open-source. Jest szeroko stosowane w dziedzinie informatyki, zarówno przez profesjonalistów ds. bezpieczeństwa, administratorów sieci, jak i entuzjastów, aby monitorować, analizować i debugować ruch sieciowy. Oto kilka kluczowych cech programu:

- Rozpoznawanie i Zbieranie Pakietów: Wireshark pozwala na przechwytywanie i analizowanie pakietów ruchu sieciowego. Może pracować na wielu platformach, takich jak Windows, macOS czy Linux.

- Wsparcie dla Wielu Protokołów: Program obsługuje wiele różnych protokołów komunikacyjnych, w tym protokoły warstwy transportowej (np. TCP, UDP), protokoły warstwy aplikacji (np. HTTP, DNS), a także protokoły bezpołączeniowe (np. Ethernet).

- Filtracja Pakietów: Wireshark umożliwia filtrację przechwyconych pakietów na podstawie różnych kryteriów, co pomaga skoncentrować się na konkretnych rodzajach komunikacji lub hostów.

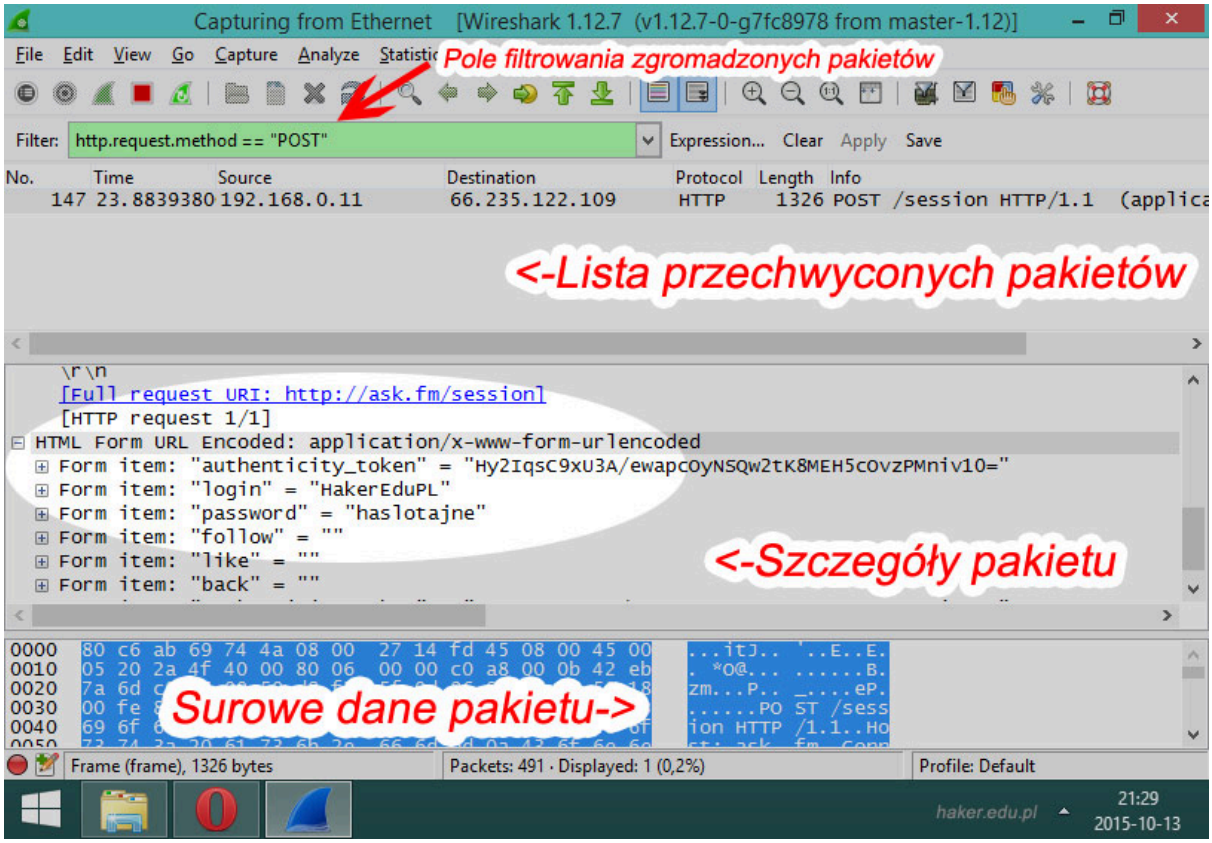
- Analiza Strumieniowa: Pozwala na analizę strumieniową danych, co jest przydatne w zrozumieniu szczegółów komunikacji między dwoma urządzeniami.

- Wygodny Interfejs Graficzny: Wireshark oferuje intuicyjny interfejs graficzny, który ułatwia korzystanie z różnych funkcji narzędzia. Dostępne są również zaawansowane opcje dla bardziej doświadczonych użytkowników.

- Statystyki: Program generuje różnego rodzaju statystyki dotyczące przechwyconego ruchu, takie jak liczba pakietów, rozkład protokołów, czy czasy odpowiedzi.

- Wsparcie dla Zaszyfrowanego Ruchu: Chociaż Wireshark sam w sobie nie jest w stanie zdekodować zaszyfrowanego ruchu, może przechwytywać zaszyfrowane dane, co może być przydatne w celu analizy ruchu.

- Społeczność i Rozszerzalność: Wireshark jest projektem open-source, co oznacza, że posiada aktywną społeczność użytkowników i deweloperów. Istnieje także możliwość rozszerzania funkcji programu poprzez dodatki.




W aplikacji znajduje się pole, w którym wpisujemy filtr, który może zastosować do znalezienia interesujących nas pakietów. Na powyższym zrzucie ekranu skorzystaliśmy z polecenia: **http.request.method == „POST”**, aby zawęzić listę zgromadzonych pakietów sieciowych.
Większą ilość gotowych filtrów możecie znaleźć w menu Analize -> Display Filters i dodatkowo w przycisku Expression.
W następnej części mamy ukazaną pełną listę zgromadzonych przez program pakietów.

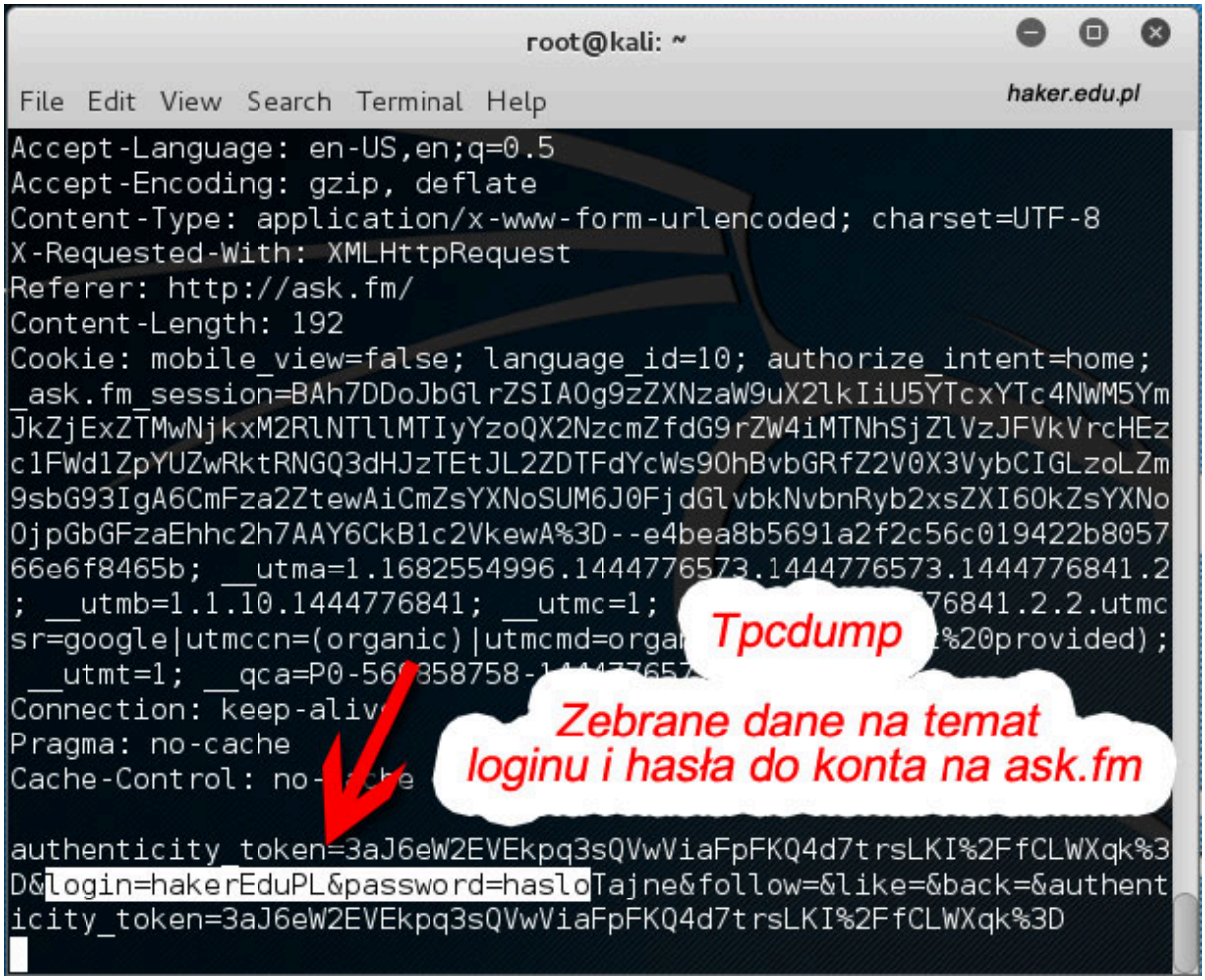
Możemy odnaleźć tutaj takie informacje o podsłuchanych danych jak:

- Time – czas o której pakiet został zarejestrowany. Format tego czasu możemy ustawić w menu Edit -> Preferences -> Columns -> Time -> Field Type. Osobiście ja polecam ustawić UTC Time,

- Source – czyli źródło z którego pochodzi pakiet. Jeżeli to my wysłaliśmy gdzieś dalej informacje (np: wchodząc na stronę www), to ten parametr będzie pokazywał nasz adres IP (ewentualnie klienta/ofiary),
- Destination – cel do którego adresowany jest pakiet. Przykład: jeżeli wchodzimy na stronę google.pl to celem będzie adres IP serwera google.
- Protocol – protokół który wykorzystywany był do transferu informacji. Polecam samemu zapoznać się z budową pakietów i doczytać czym jest sam protokół,
- Lenght – wielkość pakietu w bajtach. Im pakiet więcej ze sobą danych niesie, tym pakiet jest obszerniejszy. Wielkość pakietu jest najczęściej określana przez specyfikacje, a duże porcje danych dzielone są na mniejsze paczki a następnie składane u odbiorcy pakietu,
- Info – krótki opis typu pakietu. Wymagana jest tutaj wiedza o pakietach i protokole TCP/IP.

W szczegółach pakietu zaznaczonych na powyższym zrzucie ekranu widzimy ładnie i przejrzyście listę przechwyconych pakietów. W tym przypadku jest to login i hasło do portalu ask.fm.
W przypadku stron z protokołami SSL/TLS jest znacznie trudniej, jednak zręczny haker będzie mógł pozyskać dane w inny sposób, np. metodą phishingową.

**Program Tcpdump:**
Tcpdump to narzędzie do analizy ruchu sieciowego w systemach Unix-like. Jest to sniffer, czyli program, który przechwytuje i analizuje pakiety przesyłane przez sieć. Tcpdump umożliwia użytkownikom monitorowanie, analizowanie i diagnozowanie ruchu sieciowego w czasie rzeczywistym.




Oto kilka kluczowych cech programu Tcpdump:

Przechwytywanie Pakietów: Tcpdump pozwala na przechwytywanie pakietów przesyłanych przez sieć. Użytkownik może określić konkretne interfejsy sieciowe, z których mają być przechwytywane pakiety.

Filtrowanie Pakietów: Tcpdump umożliwia stosowanie filtrów, aby ograniczyć zakres przechwytywanych pakietów. Można definiować filtry na podstawie różnych kryteriów, takich jak adresy IP, porty, protokoły itp.

Wygodny Format Wyjściowy: Tcpdump wyświetla przechwycone pakiety w czytelny sposób, umożliwiając zrozumienie struktury pakietów oraz ich zawartości. Domyślnie prezentuje informacje w formie tekstowej, co ułatwia analizę.

Wsparcie dla Różnych Protokołów: Program obsługuje wiele protokołów, takich jak TCP, UDP, ICMP, IP, ARP, SNMP, HTTP, DNS, SSL, itp. Dzięki temu użytkownicy mogą analizować pakiety z różnych warstw modelu ISO/OSI.

Możliwość Zapisu do Pliku: Tcpdump pozwala na zapisywanie przechwyconych danych do pliku, co umożliwia późniejszą analizę lub archiwizację ruchu sieciowego.

Obsługa Sondowania Sieci: Program może być używany do sondowania sieci, sprawdzania dostępności hostów, badania tras pakietów i diagnozowania problemów związanych z ruchem sieciowym.

Przykład użycia Tcpdump:

```
tcpdump -i eth0 tcp port 80
```

**Program Ettercap:**

Ettercap to narzędzie do przeprowadzania ataków na warstwę 2 i 3 modelu OSI (warstwa łącza danych i warstwa sieci) w celu analizy ruchu sieciowego, przechwytywania pakietów, a także ataków typu "man-in-the-middle" (MITM). Program ten działa głównie w środowisku Unix-like, takim jak Linux.

Oto kilka cech i funkcji programu ettercap:

Sniffing (Nasłuchiwanie): Ettercap umożliwia przechwytywanie i analizowanie ruchu sieciowego, co obejmuje pakietowanie informacji takie jak dane logowania, hasła, czy inne wrażliwe informacje przesyłane w sieci.

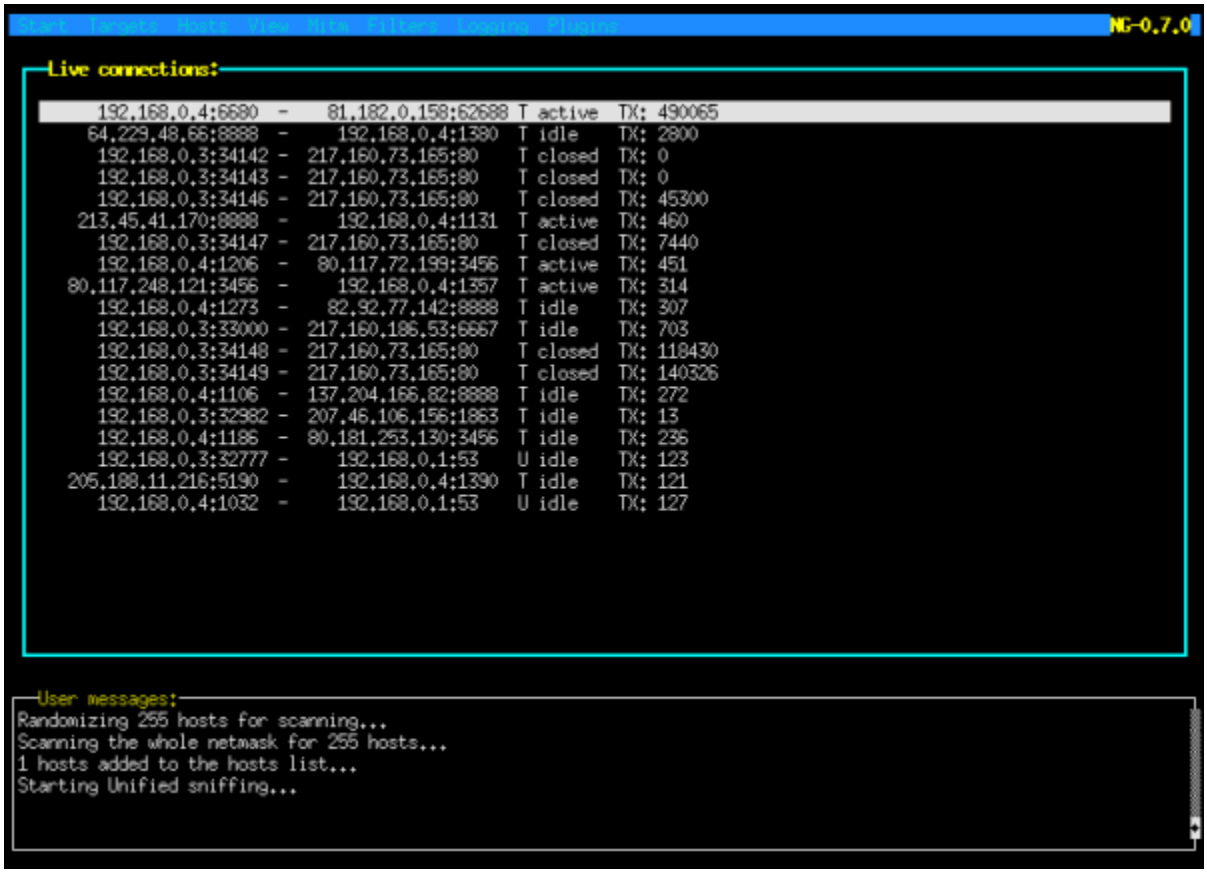
Ataki "Man-in-the-Middle" (MITM): Ettercap jest znany ze swojej zdolności do przeprowadzania ataków MITM, gdzie atakujący umieszcza się między dwoma komunikującymi się stronami, przechwytując i modyfikując przesyłane dane. To narzędzie potrafi wykonywać tego typu ataki na różnych warstwach protokołów.

Wsparcie dla Różnych Protokołów: Ettercap obsługuje różne protokoły, takie jak ARP (Address Resolution Protocol), ICMP (Internet Control Message Protocol), DHCP (Dynamic Host Configuration Protocol), SSL (Secure Sockets Layer) i wiele innych.

Filtrowanie i Modyfikowanie Pakietów: Program pozwala na filtrowanie ruchu na podstawie różnych kryteriów, co umożliwia analizę wybranych pakietów. Ponadto ettercap pozwala na modyfikację treści przesyłanych pakietów.

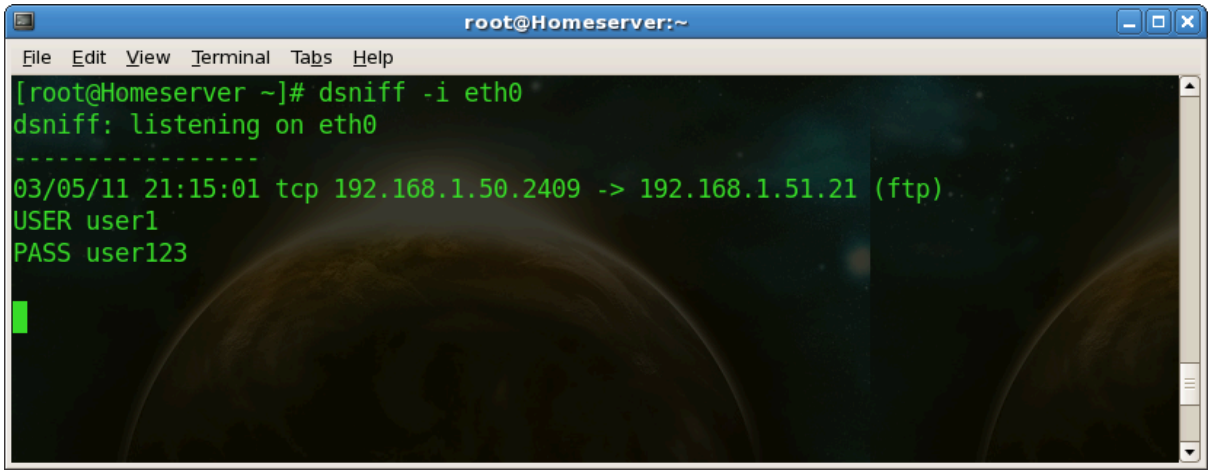
Wsparcie dla Plug-inów: Ettercap obsługuje plug-iny, co oznacza, że użytkownicy mogą rozbudowywać funkcjonalność programu poprzez dodawanie własnych skryptów i modułów.

Interfejs Graficzny i Konsolowy: Ettercap dostarcza zarówno interfejsu graficznego (GTK) jak i interfejsu konsolowego, co umożliwia użytkownikom wybór preferowanego środowiska pracy.




Wsparcie dla Systemów Operacyjnych: Chociaż ettercap jest pierwotnie przeznaczony dla systemów Unix-like, takich jak Linux, to dostępne są również wersje dla innych systemów operacyjnych, na przykład dla systemu Windows.

**Program Dsniff:**
dsniff to zbiór narzędzi do analizy ruchu sieciowego, który jest używany do przechwytywania haseł i innych poufnych informacji przesyłanych przez sieć. Program ten został stworzony z myślą o testowaniu bezpieczeństwa sieci, jednak może być także potencjalnie wykorzystywany do celów nielegalnych, dlatego jego użycie powinno być zgodne z przepisami prawnymi oraz zasadami etycznymi.




Oto kilka cech i funkcji narzędzi zawartych w pakiecie dsniff:

- arpspoof: Pozwala na przeprowadzanie ataków ARP spoofing, co umożliwia atakującemu przekierowanie ruchu sieciowego między dwoma hostami, a następnie przechwycenie i analizę danych przesyłanych między nimi.

- dnsspoof: Narzędzie to pozwala na fałszowanie odpowiedzi DNS, co może prowadzić do przekierowywania ruchu internetowego na fałszywe strony. Jest to przydatne narzędzie do ataków typu "man-in-the-middle" (MITM) z wykorzystaniem manipulacji systemem nazw domen.

- filesnarf: Przechwytuje pliki przesyłane przez sieć, co może obejmować dokumenty, obrazy czy inne typy plików.

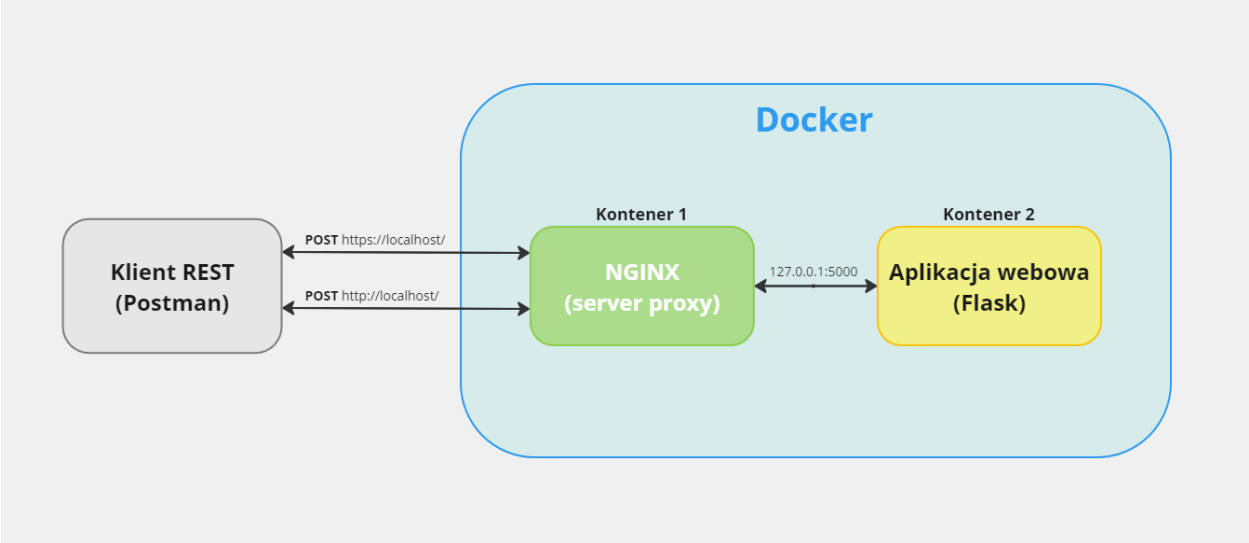
- mailsnarf: Służy do przechwytywania wiadomości e-mail przesyłanych przez sieć, co pozwala na dostęp do treści e-maili.

- msgsnarf: Przechwytuje wiadomości tekstowe przesyłane przez sieć, takie jak komunikatory internetowe.

- sshmitm: Pozwala na przechwytywanie sesji SSH, co może prowadzić do uzyskania dostępu do poufnych informacji przesyłanych w trakcie komunikacji z serwerem SSH.

# 9. Opis stworzonej infrastruktury

Stworzyliśmy prostą aplikację napisaną w Pythonie, z użyciem frameworka Flask, której zadaniem jest odebranie pliku oraz zapisanie go na dysku. Jako serwer proxy użyliśmy Nginx, który robi przekierowanie na naszą aplikację oraz szyfruje nasze połączenie. Całą konfigurację umieściliśmy w oprogramowaniu Docker, co umożliwia nam łatwe i szybkie uruchomienie całej infrastruktury, bez zbędnej konfiguracji oraz naruszania systemu operacyjnego czy tworzenia maszyn wirtualnych.




## 9.1. Docker

Docker to platforma open-source, która ułatwia automatyzację procesów tworzenia, wdrażania i uruchamiania aplikacji w kontenerach. Kontenery są lekkimi, przenośnymi jednostkami oprogramowania, które zawierają wszystko, co jest potrzebne do uruchomienia aplikacji, w tym kod, środowisko wykonawcze, biblioteki i zależności. Dzięki Dockerowi można zapakować aplikację razem ze wszystkimi jej zależnościami w jednym kontenerze, co sprawia, że jest ona przenośna i jednolita na różnych środowiskach. Używa systemu konteneryzacji, który opiera się na technologii kontenerów Linuxa, takich jak cgroups i namespaces, aby dostarczyć izolację i przenośność aplikacji. Docker jest powszechnie stosowany w środowiskach deweloperskich, testowych i produkcyjnych do automatyzacji procesów CI/CD (Continuous Integration/Continuous Deployment) oraz do zarządzania mikroserwisami.

Kontenery zapewniają izolację aplikacji od otaczającego je środowiska, co eliminuje konflikty zależności i sprawia, że aplikacja działa jednolicie niezależnie od systemu operacyjnego gospodarza.

Kontenery są przenośne między różnymi środowiskami, co ułatwia przenoszenie aplikacji pomiędzy różnymi etapami cyklu życia oprogramowania, od lokalnego środowiska deweloperskiego po środowiska produkcyjne w chmurze.

Docker pozwala na łatwe skalowanie aplikacji, ponieważ można uruchamiać wiele kontenerów na jednym host-cie.

Kontenery są lekkie i udostępniają wspólne zasoby systemowe, co pozwala na szybsze uruchamianie i efektywne wykorzystanie zasobów sprzętowych.

Docker umożliwia kontrolowanie i zarządzanie zależnościami aplikacji, co ułatwia utrzymanie spójności w różnych środowiskach.

## 9.2. Nginx

Nginx to popularny serwer HTTP oraz serwer proxy o otwartym kodzie źródłowym, który został zaprojektowany z myślą o obsłudze dużego obciążenia i zapewnianiu wysokiej wydajności. W ramach swoich funkcji, Nginx może pełnić rolę serwera plików statycznych, obsługiwać żądania klientów jako serwer HTTP, a także działać jako serwer proxy dla różnych protokołów, takich jak HTTP, HTTPS, SMTP, POP3, i IMAP.

Jedną z głównych cech Nginx jest jego zdolność do działania jako proxy odwrócony, przekierowujący ruch do serwerów backendowych. To umożliwia równoważenie obciążenia, optymalizację oraz skalowanie aplikacji. Nginx jest również efektywnym serwerem plików statycznych, obsługując bezpośrednio żądania klientów dotyczące plików graficznych, arkuszy stylów czy skryptów JavaScript.

Pod względem konfiguracji, Nginx używa czytelnych dla człowieka plików tekstowych, co ułatwia dostosowywanie zachowania serwera. Znakomita obsługa wielu równoczesnych połączeń sprawia, że Nginx jest szczególnie przydatny w sytuacjach, gdzie serwer musi obsługiwać duży ruch, takich jak popularne strony internetowe.

Dodatkowo, Nginx oferuje mechanizm przepisywania adresów URL, co pozwala dostosować strukturę adresów URL oraz przekierowywać żądania. Istotnym aspektem jest także wsparcie dla protokołu WebSocket, umożliwiającego dwustronną komunikację w czasie rzeczywistym.

Nginx znajduje zastosowanie w kontenerowych środowiskach, takich jak Docker, gdzie pełni funkcję proxy dla mikroserwisów. Istnieje również wersja komercyjna o nazwie NGINX Plus, która dostarcza dodatkowe funkcje i wsparcie techniczne.

Dzięki swojej wszechstronności, Nginx cieszy się dużą popularnością w społeczności deweloperów i jest powszechnie używany do obsługi ruchu sieciowego, równoważenia obciążenia oraz dostarczania treści internetowych w sposób efektywny i niezawodny.

## 9.3. Protokoły HTTP

HTTP, czyli Hypertext Transfer Protocol, to protokół komunikacyjny używany w architekturze klient-serwer do przesyłania danych w World Wide Web. Jest to fundament internetowej komunikacji między przeglądarkami internetowymi a serwerami, umożliwiający pobieranie stron internetowych, wysyłanie formularzy, pobieranie plików i wiele innych operacji.

Protokół HTTP jest bezstanowy, co oznacza, że każde żądanie i odpowiedź są niezależne od poprzednich. To podejście sprawia, że każde żądanie traktowane jest jako nowe, bez zapamiętywania informacji o poprzednich żądaniach. Dzięki temu rozwiązaniu, serwery HTTP mogą obsługiwać duże ilości równoczesnych połączeń.

Komunikacja oparta na protokole HTTP składa się z żądań (request) wysyłanych przez klienta do serwera oraz odpowiedzi (response) przesyłanych z powrotem przez serwer do klienta. Żądania zawierają różne metody, takie jak GET do pobierania danych, POST do przesyłania danych od klienta do serwera, PUT do aktualizacji danych, DELETE do usuwania danych i inne.

Protokół HTTP działa na modelu żądanie-odpowiedź. Klient inicjuje żądanie, a serwer odpowiada na nie, zwykle dostarczając treść strony internetowej, plik lub inną żądaną informację. Komunikacja ta odbywa się poprzez zdefiniowane nagłówki (headers), które zawierają informacje o żądaniu, takie jak metoda, adres URL, typ treści, a także nagłówki odpowiedzi, które zawierają informacje o samym zasobie, kodzie odpowiedzi HTTP, itp.

Współcześnie najczęściej stosowaną wersją protokołu HTTP jest HTTP/1.1, choć istnieje już również nowsza wersja - HTTP/2, która wprowadza szereg usprawnień związanych z wydajnością, takie jak obsługa wielu żądań jednocześnie na jednym połączeniu. Ponadto, coraz większe znaczenie zyskuje także HTTPS, czyli zabezpieczona wersja HTTP, która używa protokołu TLS/SSL do szyfrowania danych, zapewniając prywatność i bezpieczeństwo komunikacji.

W sumie, protokoły HTTP stanowią kluczowy element współczesnego internetu, umożliwiając interakcję między klientami a serwerami i umożliwiając dostęp do zasobów

online. Ich ewolucja, w tym wprowadzenie nowych wersji i zabezpieczeń, odzwierciedla dynamiczny rozwój środowiska internetowego.

## 9.4. Protokół POST, zasady jego działania

Metoda HTTP POST stanowi niezwykle istotny element protokołu HTTP, pełniąc kluczową rolę w komunikacji między klientem a serwerem w ramach World Wide Web. Postępując zgodnie z modelem żądanie-odpowiedź, metoda POST umożliwia klientowi przesyłanie danych do serwera, zwykle w celu aktualizacji zasobów, przesyłania formularzy czy tworzenia nowych zasobów.

Metoda POST różni się od metody GET, ponieważ dane przesyłane są w ciele żądania, a nie za pomocą adresu URL. Dzięki temu, metoda POST pozwala na przesyłanie większych ilości danych oraz danych wrażliwych, takich jak hasła czy informacje karty kredytowej, bez ich ujawniania w adresie URL.

W praktyce, metoda POST znajduje zastosowanie w różnych scenariuszach. W formularzach HTML, gdy użytkownik wprowadza dane i naciska przycisk "Submit", te dane są zazwyczaj przesyłane do serwera za pomocą metody POST. W rezultacie, serwer może otrzymać, przetworzyć i zapisywać te dane, na przykład do bazy danych.

W przypadku tworzenia aplikacji internetowych, metoda POST jest również wykorzystywana do wysyłania żądań do interfejsów programowania aplikacji (API), co umożliwia komunikację między różnymi serwerami lub aplikacjami. To podejście pozwala na przekazywanie różnych rodzajów danych, takich jak JSON lub XML, w celu wykonania określonych operacji.

W kontekście bezpieczeństwa, szczególnie w przypadku przesyłania wrażliwych danych, zaleca się korzystanie z protokołu HTTPS zamiast HTTP, co zapewnia szyfrowanie danych przesyłanych między klientem a serwerem, minimalizując ryzyko przechwycenia czy podsłuchiwania informacji.

Podsumowując, metoda POST stanowi istotny mechanizm w komunikacji HTTP, umożliwiając bezpieczne i efektywne przesyłanie danych między klientami a serwerami, co jest kluczowe w funkcjonowaniu dynamicznego i interaktywnego środowiska internetowego.

# 10. Opis doświadczenia

Eksperyment, który przeprowadziliśmy, skoncentrował się na przesłaniu pliku na serwer, wykorzystując zarówno komunikację szyfrowaną, jak i nieszyfrowaną, w celu

dokładnego porównania obu sytuacji. W celu osiągnięcia tego celu, specjalnie przygotowaliśmy plik testowy o nazwie test.txt, który zawierał niewielką ilość danych. Procedura eksperymentalna rozpoczęła się od wysłania żądania POST do naszego serwera, wraz z przesyłanym plikiem.

W trakcie tego procesu monitorowaliśmy ruch sieciowy za pomocą programu WireShark, który precyzyjnie zarejestrował całą aktywność komunikacyjną. Naszym celem było zidentyfikowanie wysłanych pakietów, które następnie poddaliśmy analizie. Kluczowym elementem badania było porównanie dwóch scenariuszy: komunikacji z zastosowaniem szyfrowania oraz bez użycia zabezpieczeń.

Analiza danych zebranych podczas eksperymentu pozwoliła nam lepiej zrozumieć różnice między transmisją danych w środowisku szyfrowanym a nieszyfrowanym. Wyniki naszych obserwacji mogą przyczynić się do lepszego zrozumienia bezpieczeństwa komunikacji online oraz potencjalnych zagrożeń związanych z brakiem zabezpieczeń.

## 10.1. Pozyskiwanie certyfikatu

Ponieważ doświadczenie przeprowadzane jest na lokalnym komputerze użyliśmy certyfikatu wygenerowanego ręcznie.

**Self-Signed Certificate** (Certyfikat Własnoręcznie Podpisany) to rodzaj certyfikatu bezpieczeństwa używanego w środowiskach informatycznych. Jest to sposób potwierdzania tożsamości serwera lub aplikacji, który nie wymaga zaangażowania zewnętrznego dostawcy certyfikatów. W przeciwieństwie do certyfikatów wystawianych przez zaufane jednostki certyfikujące (CA - Certificate Authority), self-signed certificate jest podpisywany przez samą jednostkę, dla której jest on przeznaczony.

Główne cechy self-signed certificate to

- **Brak trzeciej strony zaufanej:**
  Self-Signed Certificate nie jest weryfikowany przez zewnętrzną jednostkę certyfikującą, co oznacza, że nie istnieje zewnętrzny organ potwierdzający tożsamość serwera czy aplikacji.
- **Samopodpisywanie:**
  Certyfikat jest generowany i podpisywany przez samą jednostkę, dla której jest przeznaczony. Klucz publiczny przypisany do certyfikatu jest sam podpisany kluczem prywatnym tej samej jednostki.
- **Proces generowania:**
  Proces generowania self-signed certificate obejmuje utworzenie pary kluczy (publicznego i prywatnego) oraz utworzenie certyfikatu, który zawiera informacje

o właścicielu certyfikatu (np. nazwa domeny), klucz publiczny oraz podpis cyfrowy wykonany przy użyciu klucza prywatnego.

- **Brak globalnego uznania:**
  Ponieważ self-signed certificate nie jest wystawiany przez zaufaną jednostkę certyfikującą, nie jest on powszechnie uznawany przez przeglądarki internetowe czy inne aplikacje, co może powodować pojawianie się komunikatów ostrzegawczych dla użytkowników.

W celu wygenerowania takiego certyfikatu użyliśmy polecenia:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/nginx-selfsigned.key \
-out /etc/ssl/certs/nginx-selfsigned.crt \
-subj
"/C=PL/ST=Malopolska/L=Cracow/O=Projekt/OU=IT/CN=example.com/ema
ilAddress=admin@example.com"
```

## 10.2. Implementacja certyfikatu

Po wygenerowaniu certyfikatu należy zmodyfikować konfigurację Nginx. Do pliku nginx.conf musimy dodać następujące parametry:

- **ssl_certificate:**
  Wskazuje ścieżkę do pliku zawierającego certyfikat SSL, który jest używany do uwierzytelniania serwera w trakcie procesu nawiązywania połączenia SSL/TLS.
- **ssl_certificate_key:**
  Określa ścieżkę do pliku zawierającego klucz prywatny serwera, który jest używany do deszyfrowania informacji przesyłanych między klientem a serwerem.
- **ssl_protocols:**
  Określa, które wersje protokołów SSL/TLS są dozwolone. Może zawierać jedną lub więcej wartości, oddzielonych spacjami. Przykładowe wartości to TLSv1.2 i TLSv1.3.
- **ssl_ciphers:**
  Określa zestawy szyfrów kryptograficznych, które są dozwolone podczas nawiązywania połączenia SSL/TLS. Jest to lista preferowanych algorytmów szyfrowania, oddzielonych dwukropkami.

Fragment pliku, który przygotowaliśmy:

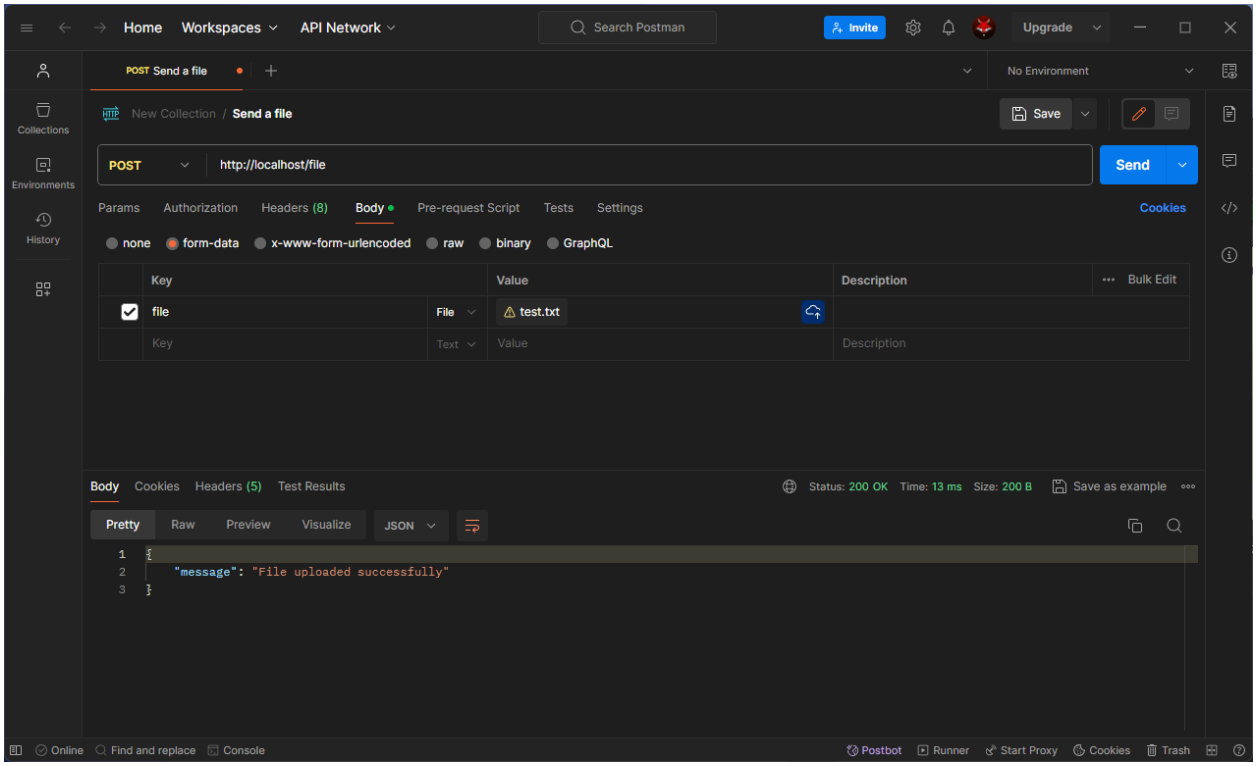
```
server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers 'TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POL

    location / {
        proxy_pass http://flask-app:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static {
        alias /app/static;
    }

    location /favicon.ico {
        alias /app/static/favicon.ico;
    }

    error_page 500 502 503 504 /500.html;
    location = /500.html {
        root /app/templates;
    }
}
```

## 10.3. Przechwytywanie danych

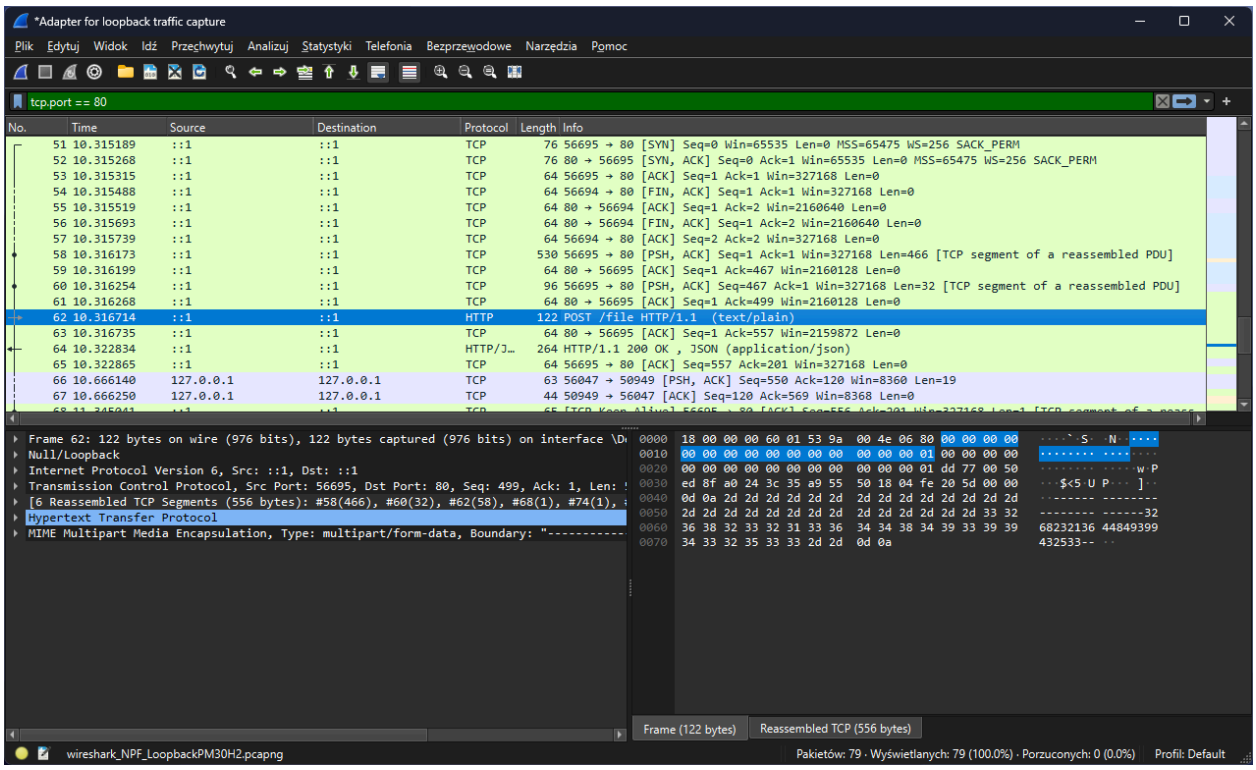
Wysyłamy dwa żądania POST do naszego serwera. Jedno używając protokołu http a drugie https, które oznaczają kolejno komuniację nie szyfrowaną i szyfrowaną. W czasie wykonywania żądania uruchomiony jest program WireShark z włączonym nasłuchiwaniem.

### 10.3.1. Bez zabezpieczenia

Wysłanie żądania:



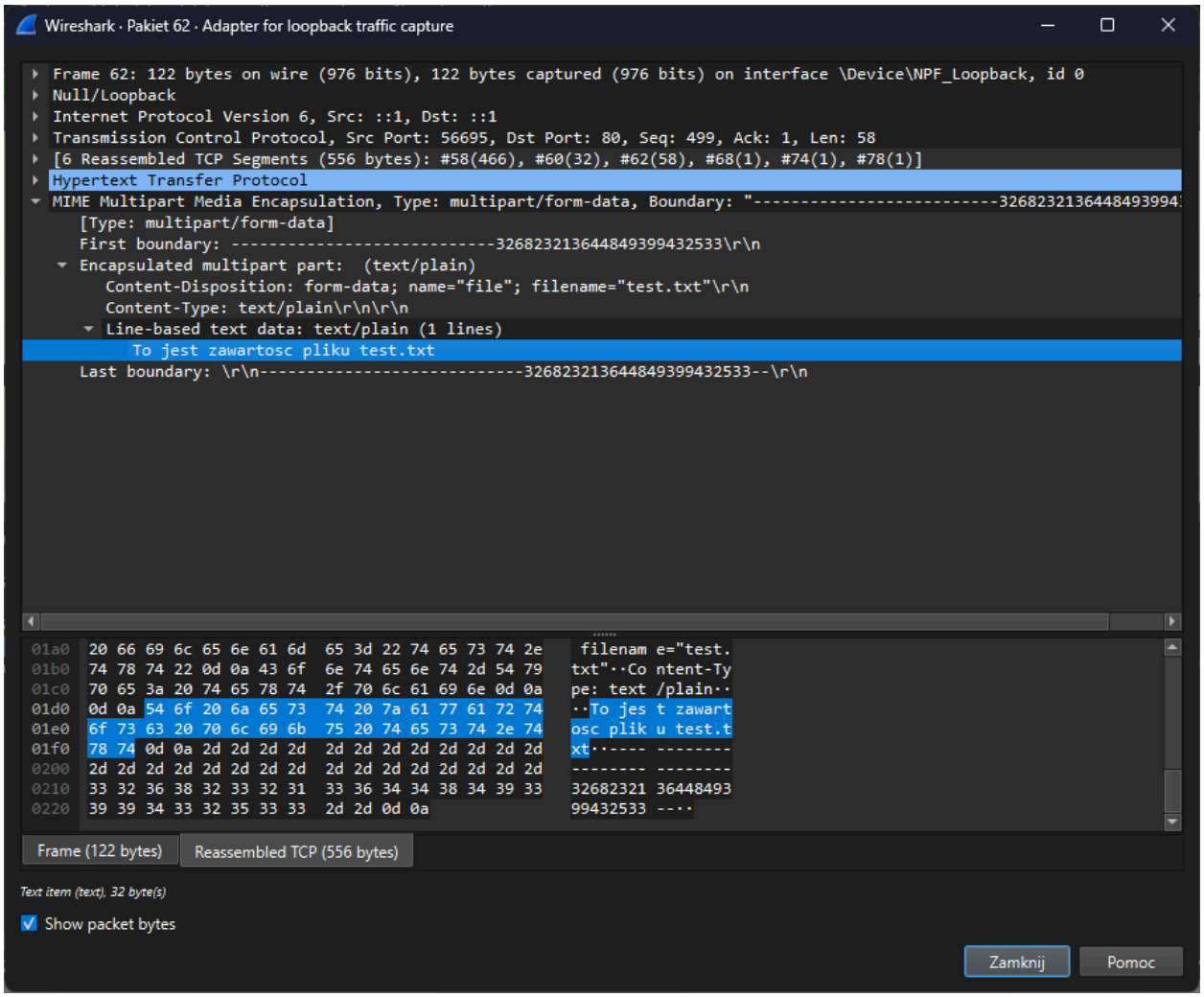

Program WireShark przechwycił następujące pakiety:



Jak widać otrzymaliśmy dosyć klarowną informację na temat przesłanego żądania. Podświetlony jest pakiet który dotyczy samego wysłania informacji, już na pierwszy rzut oka widać, na jaki endpoint została wysłana informacja oraz co zawierała - protokół MIME (Multipurpose Internet Mail Extensions).

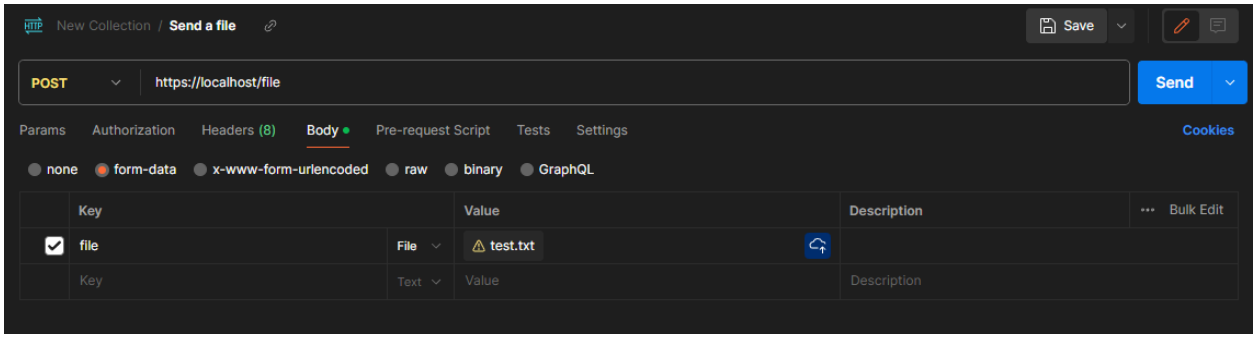
Jeżeli przyjrzymy się bliżej, rozwijając informację o przesłanych danych, to będziemy w stanie odczytać więcej szczegółów. Ukazują nam się dane takie jak:

- **Content-Disposition:** nazwa przesyłanego pliku
- **Content-Type:** rodzaj pliku, w naszym przypadku jest to plik tekstowy
- **Line-based text data:** zawartość przesłanego pliku

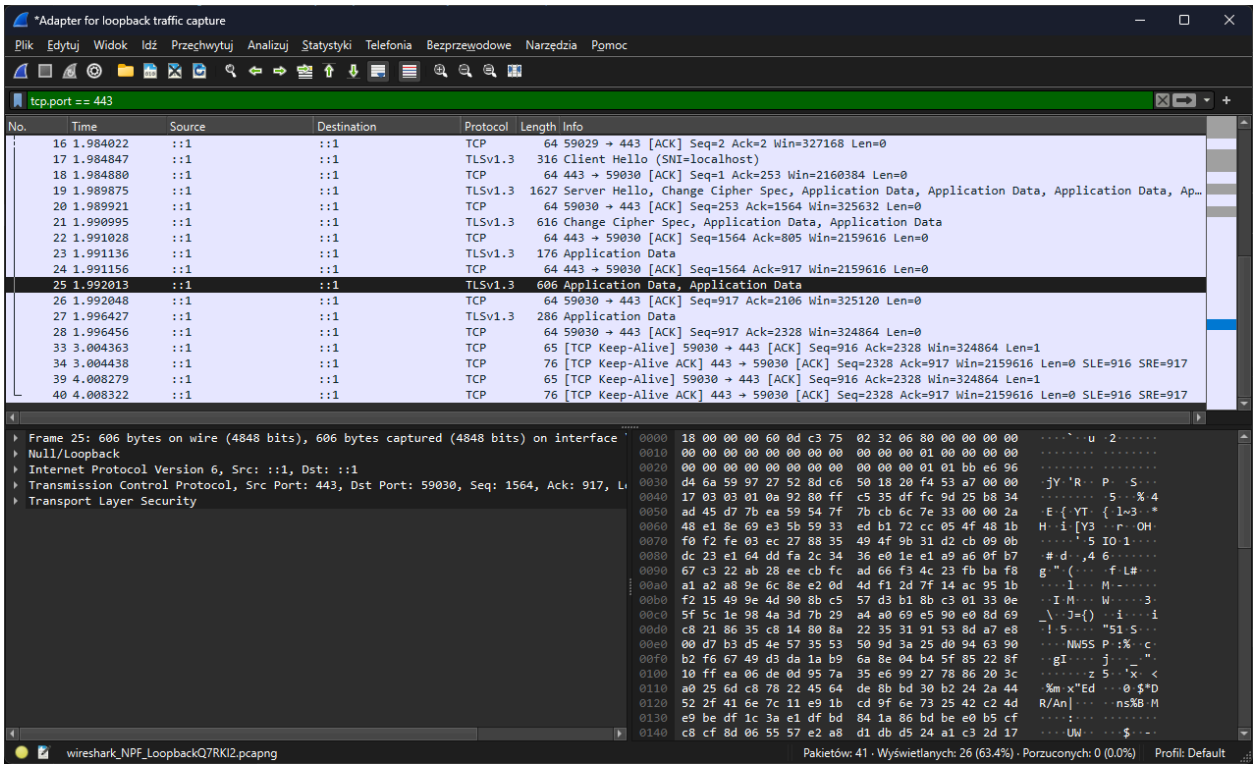



## 10.3.2. Z zabezpieczeniem

Wysłanie żądania wygląda podobnie jak w punkcie wyżej (10.3.1). Ulega zmianie tylko protokół **http** na **https**.




Program WireShark przechwycił następujące pakiety:




Jak widać nie ma już widocznych informacji które by coś dla nas znaczyły. Zauważyć można pakiety które są charakterystyczne dla protokołu TLS. Pakiety odpowiedzialne za dostarczenie danych do serwera kryją się pod **Application Data**, ale nasze informacje kończą się tutaj. Nic więcej nie jesteśmy w stanie się dowiedzieć na temat tego żądania.

### 10.3.3. Z zabezpieczeniem, ale posiadając klucze certyfikatu

Odszyfrowanie ruchu SSL/TLS za pomocą Wiresharka może być wyzwaniem, ponieważ standardowo komunikacja ta jest szyfrowana w celu zapewnienia prywatności i bezpieczeństwa. Jednakże, jeśli posiadamy dostęp do kluczy prywatnych używanych przez serwer, możemy użyć ich w Wiresharku do odszyfrowania ruchu SSL/TLS. Należy

pamiętać, że próba odszyfrowania ruchu SSL/TLS bez zgody i właściwych uprawnień jest nielegalna i narusza prywatność.

**Instrukcja krok po kroku:**

1. Uzyskajmy klucz prywatny
2. Uruchommy Wiresharka i przejdźmy do menu "Edit" (Edycja) > "Preferences" (Preferencje).
3. Dodajmy klucz prywatny. W preferencjach Wiresharka przejdźmy do sekcji "Protocols" (Protokoły) > "TLS" > "RSA Keys List" (Lista kluczy RSA). Następnie kliknijmy przycisk "Edit" (Edytuj), a potem "+" (Znak plusa). Wprowadźmy adres IP serwera, port oraz ścieżkę do pliku z kluczem prywatnym.
4. Po dodaniu klucza prywatnego kliknijmy "OK", aby zapisać zmiany w preferencjach.
5. Uruchamiamy przechwytywanie ruchu w Wiresharku, aby zacząć rejestrować pakiety SSL/TLS.
6. Po zarejestrowaniu ruchu, możemy analizować pakiety SSL/TLS. Wireshark powinien automatycznie odszyfrować ruch, który został zarejestrowany przy użyciu dodanego klucza prywatnego. Te pakiety wyglądają identycznie jak w punkcje **10.3.1**

## 10.4. Podsumowanie

Analiza ruchu za pomocą WireSharka podkreśla konieczność implementacji SSL w celu skutecznego zabezpieczenia danych online. Jest to nie tylko kwestia ochrony prywatności, ale również o zapobieganie potencjalnym zagrożeniom związanym z manipulacją danymi czy atakami na integralność informacji.

W przypadku korzystania z SSL, dane są zaszyfrowane, co oznacza, że nawet w trakcie skanowania WireSharkiem trudno jest odczytać ich zawartość – brak szczegółowych informacji. Szyfrowanie to stanowi istotną barierę dla potencjalnych hakerów, chroniąc poufne dane przed przechwyceniem i manipulacją.

W sytuacji braku SSL, skanowanie WireSharkiem może ujawnić pełną zawartość przesyłanych danych, stanowiąc potencjalne niebezpieczeństwo dla ich poufności. Bez zabezpieczenia szyfrowania, informacje stają się narażone na ataki, a użytkownicy są bardziej podatni na utratę prywatności oraz możliwe oszustwa.

# 11. Instrukcja jak odtworzyć doświadczenie

Odtworzenie doświadczenia jest proste i nie zajmie dużo czasu.

**Potrzebne oprogramowanie**

- Docker (wraz z Docker Compose): https://www.docker.com/
- WireShark: https://www.wireshark.org/

**Uruchomienie konfiguracji**

1. Rozpakowywujemy **ssl_tls.zip**, który jest dołączony do projektu
2. Uruchomienie poleceń w głównym katalogu projektu. Polecenia te zbudują nam obrazy z aplikacją oraz serwerem, następnie uruchomią kontenery na podstawie tych obrazów, które będą działać w tle.
    - `docker-compose build`
    - `docker-compose up -d`
3. Uruchamiamy WireShark i rozpoczynamy nasłuchiwanie w **Adapter for loopback traffic capture**
4. Wysyłamy nasze żadania na adres **https://localhost/file** oraz **http://localhost/file**
5. Analizujemy przechwycony ruch
6. Zatrzymujemy kontenery z aplikacją i serwerem
    - `docker-compose down`